

3 第三章 英雄集结：单机并发

第二章中，我们观察了一次在线请求的完整处理过程：客户端通过网络提交输入，服务器完成校验与状态裁决，再把结果返回相关客户端并保存关键记录。这一阶段关注的是少量请求能否正确完成。当客户端和请求继续增加、系统仍然运行在一台服务器上时，即使网络读写已经与世界主循环分开，服务器仍需在有限时间内逐个完成事件校验、状态更新和结果生成，内部处理路径因而开始承受压力。

更多事件进入同一条内部处理路径后，数据库访问等慢操作和复杂规则计算会延长单个事件的处理时间，后续事件随之在队列中等待，尾延迟不断上升。为了让网络读写、外部访问和彼此独立的计算任务能够同时推进，系统需要引入并发执行；多个执行流又可能同时读写共享状态，打乱原本确定的检查与修改顺序。连接和临时对象的频繁创建、慢速存储的反复访问，还会持续消耗 CPU、内存和网络资源，使单机承载能力提前触顶。这些问题同样存在于即时通信、在线协作、电商交易和模型服务中。

本章要解决的问题是：在不增加机器数量的前提下，如何组织并发执行、保护共享状态并提高资源利用率，使在线系统能够承载更多请求，同时判断单机优化何时接近上限。首先区分等待型任务与计算型任务，说明进程、线程、协程、事件驱动与通道（channel）如何组织执行；随后讨论多个执行流访问共享状态时的竞态、互斥和等待通知；最后说明连接池、对象池和缓存分层如何减少资源开销，并用性能观测验证优化效果。本章以并发隔离等待、同步保护共享状态、资源复用降低开销为主线。在线游戏中的多人同图、非玩家角色（Non-Player Character, NPC）奖励和状态同步是贯穿全章的教学案例，但这些机制同样适用于即时通信、电商交易、协作文档和模型服务等在线系统。

3.1 从顺序处理到并发组织

低负载下，在线系统可以沿一条顺序路径依次完成接入、处理、状态访问和结果返回，代码简单且容易验证。请求数量增加后，多个连接和任务会在相近时间进入系统，顺序路径首先暴露两类不同性质的问题。第一类来自等待：某个客户端网络延迟、一次数据库查询或一次缓存访问，都可能让后续请求停在队列中。第二类来自计算：内容编码、规则计算、数据过滤等工作会持续占用 CPU，单核算力不足时也会形成积压。多人在线场景可以具体呈现这两类压力：连接读写和外部查询带来等待，碰撞检测、AOI（感兴趣区域）过滤和批量序列化消耗算力。同样的两类压力也出现在电商大促的下单峰值（订单查询与库存校验等待，反作弊规则与价格计算消耗算力）、实时协作的消息同步（消息分发等待，权限校验与渲染任务消耗算力）等场景中。等待问题需要让不同任务在时间上交错推进，计算问题则需要把可拆分的计算段分摊到多个 CPU 核上。用更概括的说法，**并发用于隔离等待传播**，让一个任务等待 I/O 时其他任务仍可推进；**并行用于分摊计算负载**，让可拆分的计算段同时使用多个 CPU 核。本节先从顺序主循环中的等待传播讲起，再区分并发与并行，并把这组分工落实到进程、线程和 goroutine 这些执行单元上。

3.1.1 顺序处理如何放大延迟

低负载下，在线系统可以用一条顺序循环依次处理请求：按到达顺序读取连接输入，完成校验与状态更新后再返回结果。由于连接数少、事件密度低，这种顺序实现通常不会立刻造成可感知的延迟。请求数量增加后，顺序结构就会把最慢的环节放大成全局问题。以游戏服务器为例，四人同图时若有一名玩家网络延迟达到 500ms，顺序阻塞读取会让整轮推进被拖慢，原本流畅的其他玩家也会被迫等待。单个慢连接的等待被主循环放大为全局停顿，这就是等待传播的排队效应。

这个例子中的瓶颈来自等待被串进同一条主处理路径。系统需要先把慢连接、缓存查询这类 I/O 等待从主循环中拆开，再把内容编码、规则计算等可拆分的计算段推到多个 CPU 核上。前一段已经把并发与并行的分工概括为“隔离等待、摊开计算”；接下来要把这条分工落实到进程、线程和 goroutine 上。对在线系统而言，慢连接、数据库访问和缓存访问属于可隔离的等待，数据过滤、规则计算和批量序列化属于可摊开的计算段，游戏服务器中的碰撞检测与第二章介绍的 AOI 过滤也属于后者。图17把这组差异放到同一时间轴上。左侧只有一条单核执行带，同一时刻只运行一个任务；某个任务进入 I/O 等待后，CPU 才切换到其他可运行任务。右侧的三个 CPU 核则在同一时间区间分别执行计算段，因此总计算可以早于单核顺序基线完成。

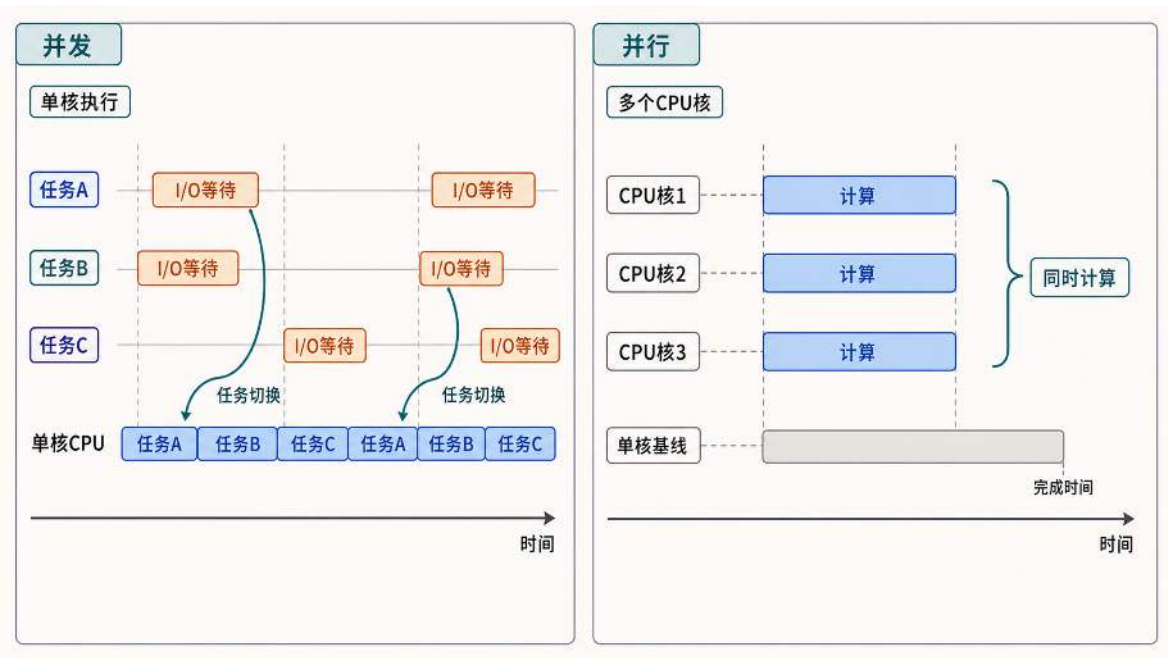


图 17: 并发用交错执行隐藏 I/O 等待，并行用多核同时计算缩短总计算时间。

当连接规模从少量请求扩展到多人同时在线时，顺序执行模型会让多种压力汇入同一条主处理队列。图18对比了请求数量少与连接规模扩大后的两种状态：双人原型下顺序处理仍能稳定推进；当规模扩大到多人同图后，连接数、事件密度、I/O 等待与热点状态访问同时增加，共同推高主处理队列的长度。队列持续变长后，后续请求会在同一条主处理路径上等待更久，尾延迟急剧上升，慢请求还会把等待传播到后续所有请求。

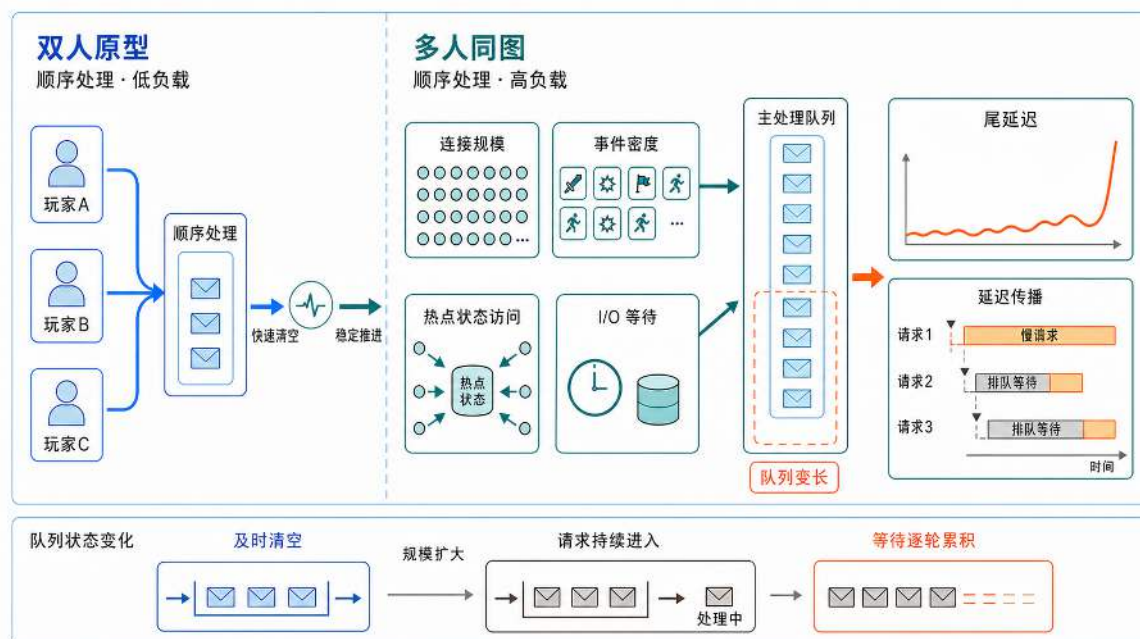


图 18: 从双人原型到多人同图，连接规模、事件密度、I/O 等待和热点状态访问共同汇入主处理队列；队列变长后尾延迟急剧上升，慢请求还会把等待传播到后续请求。

图18中的四类压力进入主处理路径后，会分别在连接维护、事件处理、外部等待和热点状态修改上形成瓶颈。需要注意的是，前三类压力是顺序模型本身就会暴露的；真正的共享状态并发冲突，要等到下一节引入并发执行流后才会成为独立问题。

第一个瓶颈是连接规模的增长。先以 50 个连接为起点：系统需要维护的长连接数（例如 WebSocket 或 TCP）从 2 个增长到 50 个，而每个连接都需要独立的读写缓冲区、心跳检测机制与超时管理逻辑。如果采用传统的一连接一线程模型，每个线程通常需要预留 MB 量级的栈虚拟地址空间（具体随操作系统默认值与配置不同），50 个连接对应的预留量就可能达到数十到数百 MB。预留地址空间不等于已经占用同等规模的物理内存，线程栈会按实际使用逐步提交，但线程数量继续增长后，地址空间、已提交栈内存和调度成本都会累积。这个数字只是教学起点；当连接数继续增长到数百、数千，甚至用来讨论上万连接的资源上限时，线程栈、创建销毁与上下文切换的成本会更快暴露出来。操作系统线程的创建和销毁涉及系统调用，上下文切换需要保存和恢复完整的 CPU 寄存器状态，这些开销在高频场景下会成为明显的性能拖累。

第二个瓶颈是事件密度的快速放大。输入事件数量大致随在线人数线性增长，但输出侧的状态推送会被广播进一步放大。假设一个实时场景中每名用户平均每秒产生 5 次操作，100 名并发用户就需要服务器处理每秒 500 次的输入事件。输出侧的压力更为严峻：每次状态变更都需要推送给相关用户，若平均每人的可见或订阅对象有 10 个，则每秒需要发送 5000 次状态同步消息。如果所有用户都关注同一份全局状态，广播开销会接近平方级；若通过兴趣过滤把推送范围稳定限制住，它更接近“人数乘以平均关注数”的线性放大。在多人游戏中，玩家移动、攻击等操作触发对视野内其他玩家的状态广播，同样面临这一放大效应。在顺序执行模型中，处理这些事件的总时间是所有单次操作时间的简单累加，当事件密度超过处理能力上限时，消息队列会迅速积

压，响应延迟从毫秒级恶化到秒级。

第三个瓶颈是 I/O 等待时间的全局传播。在单机服务器中，许多操作都涉及 I/O 等待：读取用户会话可能要等待一次缓存服务的网络往返，查询订单信息可能要等待数据库或磁盘路径，向客户端发送响应也可能要等待 TCP 发送缓冲区就绪。在顺序执行模型中，主线程在等待某条连接的外部查询响应时会完全阻塞，无法处理其他连接的请求。这意味着一个慢 I/O 会传播到所有后续请求，造成全局性的响应延迟增长。即使系统配备了高性能 CPU 和大容量内存，这些计算资源也会因为 I/O 等待而长时间处于空闲状态，资源利用率极低。

第四个瓶颈会在引入并发后出现：共享资源的并发访问冲突。当多个用户几乎同时操作同一资源（如抢购限量商品、编辑同一份文档的同一行、向同一账户并发转账）时，系统必须确保操作的原子性和结果的确定性。在顺序主循环中，所有操作按到达顺序依次执行，并发冲突不会出现。但当系统为了解决前三个瓶颈而引入并发处理后，如果没有配套的同步机制，就可能出现数据竞争：两条路径同时通过检查条件并执行修改，导致重复扣减或重复发放。以游戏为例，两名玩家同时拾取地图上唯一的宝箱，若无同步保护，就可能都认为自己成功，破坏规则公平性。这类问题的根源在于检查和修改操作之间存在未受保护的间隔，多个执行流可能同时通过检查条件并执行修改操作。

从更整体的视角看，延迟传播背后通常是排队效应。请求到来的速度一旦接近系统的处理速度，哪怕只是少量抖动，也会被队列放大成更长的等待。此时，平均延迟可能还维持在表面可接受的范围里，尾延迟已经被少数慢请求拉高：后续请求被挡在队列里，用户感知到的卡顿或超时更集中地落在尾部分请求上。回到并发与并行的分工：前者把等待拆开，后者把算力摊开，目的都是在有限资源内缩短这条排队链条。

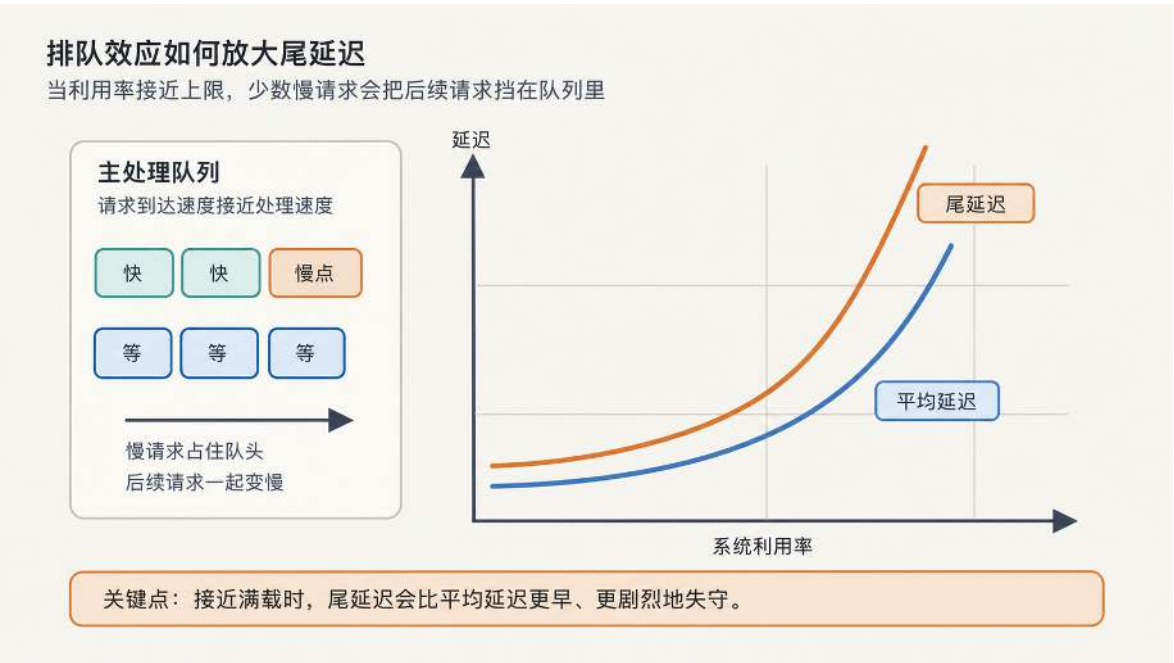


图 19: 利用率接近上限后，少量到达率抖动会被队列放大为尾延迟的非线性恶化。

图19强调的是接近满载时的非线性变化。只要请求到达速度接近服务器处理速度，一个慢请

求占住队头，后续请求就会一起等待。平均延迟可能只是逐步上升，但尾延迟会更早、更剧烈地拉开。这类积压关系可以用 Little 定律 $L = \lambda W$ 描述： L 表示系统中平均积压的请求数， λ 表示请求到达率， W 表示请求从进入系统到处理完成的平均停留时间。当慢请求把 W 拉长时，在相同到达率下，队列中的积压请求数 L 也会增加；积压越多，锁竞争、缓存抖动和批量输出就越容易继续放大延迟。

这些瓶颈也会出现在其他在线服务中。即时通讯、金融交易、协同办公这类在线服务，只要进入大量用户同时在线的阶段，都会遇到类似的资源组织问题：多路连接怎么管理，高频事件怎么消化，I/O 等待怎么隔离，共享状态怎么保护。并发编程的目标，是在给定硬件条件下控制排队长度，让吞吐和尾延迟都留在可接受的范围里。

3.1.2 操作系统的抽象：进程与线程

上一节把顺序主循环如何放大延迟拆解清楚了：等待要拆开，队列要压住。要实现这些，首先需要操作系统提供的执行组织方式。以采用固定节拍推进状态的实时服务为例，一个进程通常同时维护多类工作：连接读写持续接收客户端输入，主事件循环推进业务状态，输出路径把状态变化推送给相关用户，持久化任务把交易结果或日志写入外部存储。这些工作最终都要交给操作系统调度执行。在操作系统提供的抽象中，进程负责组织资源和隔离故障，线程负责承载可被 CPU 调度的执行流。

进程可以理解为一个被操作系统托管的服务实例。在线服务进程拥有自己的地址空间、文件句柄、网络连接和权限信息；网关进程、日志进程和其他业务进程也各自拥有独立的运行环境。这种隔离让故障影响范围变得可控：一个业务进程崩溃时，该实例服务的用户会全部受影响，网关、日志服务和其他业务进程仍可继续运行。

线程位于进程内部，负责执行具体代码。操作系统调度器真正放到 CPU 核上运行的是线程。每个线程有自己的寄存器上下文和调用栈，用来记录当前执行到哪一步；同一进程内的多个线程共享堆内存、文件句柄和网络连接。共享内存让线程之间传递数据更方便，例如连接线程可以把解析出的客户端输入交给业务逻辑线程；这种共享也要求程序写清同步规则，否则多个线程同时读写共享订单、会话状态或推送队列时，结果就会变得不可预期。

这组区别会影响在线服务的组织方式。对在线服务而言，进程主要划定服务实例和故障影响范围，线程则承载连接读写、输入处理和状态推送等可被 CPU 调度的工作。

表 1: 进程与线程在在线服务中的职责差异

对比维度	进程 (Process)	线程 (Thread)
在在线服务中的角色	承载业务服务、网关或日志服务等进程实例	在同一服务实例内执行连接读写、输入处理、状态推送等工作
资源归属	拥有独立地址空间、文件句柄、网络连接和权限信息	共享所属进程的堆内存、文件句柄和网络连接，保留独立栈与寄存器上下文
故障影响	单个业务进程崩溃时，影响主要集中在该实例服务的用户范围	线程出错可能破坏同一进程内的共享状态，严重时导致整个进程退出
并发代价	隔离更强，但跨进程通信和实例切换成本更高	共享数据更方便，但需要同步规则保护共享状态
适用场景	适合划分服务实例和故障影响范围	适合承载实际执行工作，数量增长会带来栈、切换和同步成本

表1说明，进程和线程解决的是两类不同问题：进程更适合划分业务服务、网关和日志服务等进程实例，并把单个实例崩溃后的影响限制在相对明确的范围内；线程更适合承载连接读写、输入处理和状态推送等会被 CPU 调度执行的工作。在一连接一线程的模型下，连接数量增加时线程数量也随之增长，会带来三类成本。第一是栈空间成本。每条线程都有独立栈，连接规模上升后，服务器除了保存会话状态和消息缓冲区，还要为大量线程保留栈空间。第二是切换成本，线程切换需要保存与恢复寄存器、更新调度结构，并可能破坏 CPU 缓存局部性。第三是同步成本，线程共享堆内存和对象引用，越多执行流并行读写同一份状态，越需要明确临界区和锁顺序。

线程数量继续增长时，成本会从“执行计算”扩展到“维持执行流”本身。在线服务在推进业务状态之外，还要长期维持连接读写、存档落库和状态推送等路径。连接读路径会停在客户端下一条输入到达之前，存档路径会停在数据库或缓存返回之前，推送路径会停在客户端连接重新可写之前。这些路径停住时，CPU 未必在执行业务计算；如果每条路径都占用一条操作系统线程，线程栈和调度实体仍然会被保留。连接数和外部访问次数继续上升后，服务器会积累大量停在返回点之前的执行流，线程模型的资源成本也会随之放大。

前面讨论进程和线程时，在线服务已经被拆成两层职责：进程划出服务实例和故障影响范围，线程承载会被 CPU 调度执行的工作。但连接读写、外部查询和状态推送这类路径经常停在等待点上，直接用线程承载会把等待成本也放大成线程成本。协程通常指由用户态运行时管理的轻量执行抽象，但不同语言对调度方式的规定并不相同。Go 中对应的执行单元称为 goroutine；它与经典的协作式协程并非完全同义，因为 Go 运行时会把 goroutine 复用到多条操作系统线程

上，并支持运行时抢占。⁸ 图20把这三层关系放在一起：进程保存资源并隔离故障，线程承接内核调度，以 goroutine 为代表的轻量执行单元把大量等待型工作复用到较少线程上。

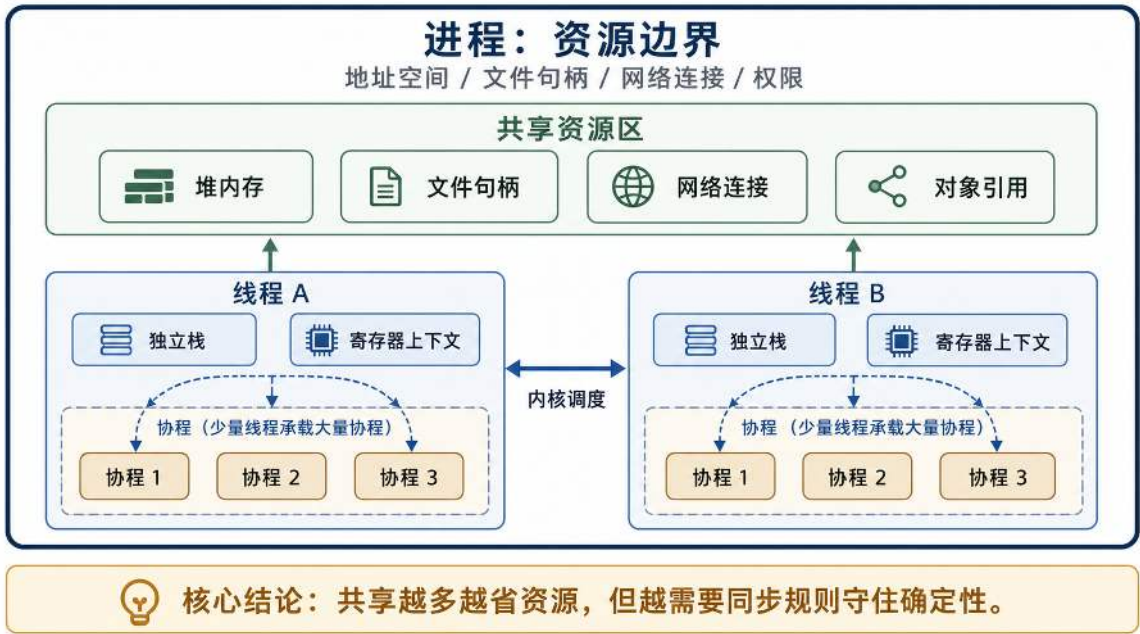


图 20: 进程、线程与轻量执行单元的三层关系：进程保存资源并隔离故障，线程承接内核调度，以 goroutine 为代表的轻量执行单元把大量等待型工作复用到较少线程上。

3.1.3 用 goroutine 组织等待型任务

在 Go 中，goroutine 是由运行时管理的轻量执行单元。它适合承载那些职责清楚、但经常在网络读写、缓存查询或数据库访问时进入等待的执行流程。放到在线服务中，连接读可以写成一个 goroutine，持续等待客户端发来的下一条输入，并把解析后的消息投递到输入队列；连接写可以写成另一个 goroutine，从发送队列取出推送结果，再等待连接可写后发回客户端；用户查询、订单结算落库这类外部 I/O 也可以放入后台 goroutine，等待缓存或数据库返回结果。

这样拆分后，代码仍然按照业务路径组织：连接读写负责网络消息进出，外部 I/O 负责访问数据库或缓存，后台推送负责处理发送缓冲区。但一个新问题随之出现：goroutine 数量远超 OS 线程时，谁来决定哪个 goroutine 占用哪个 OS 线程？

Go 运行时的调度器承担这件事。数量较多的 goroutine 先进入可运行队列，调度器再把其中若干个映射到较少的 OS 线程上执行。当 goroutine 在 channel 或运行时可以管理的网络 I/O 上等待时，它会被挂起，原来承载它的 OS 线程可以继续执行其他已就绪的 goroutine。若 goroutine 进入真正阻塞的系统调用或 cgo 调用，原 OS 线程仍可能被占用；运行时安排其他线程承载可运行的 goroutine。这种把 M 个 goroutine 复用到 N 个 OS 线程（N 通常远小于 M）的策略常称为 **M:N 调度**。它降低了等待型工作与操作系统线程的一一绑定：连接规模扩大时，OS 线程

⁸Go 官方文档对 goroutine 与操作系统线程复用关系的说明见 https://go.dev/doc/effective_go；Go 1.14 引入异步抢占的说明见 <https://go.dev/doc/go1.14>。

数量通常不必随连接数按比例增长，等待期间也不必为每条路径长期保留一条独立线程。

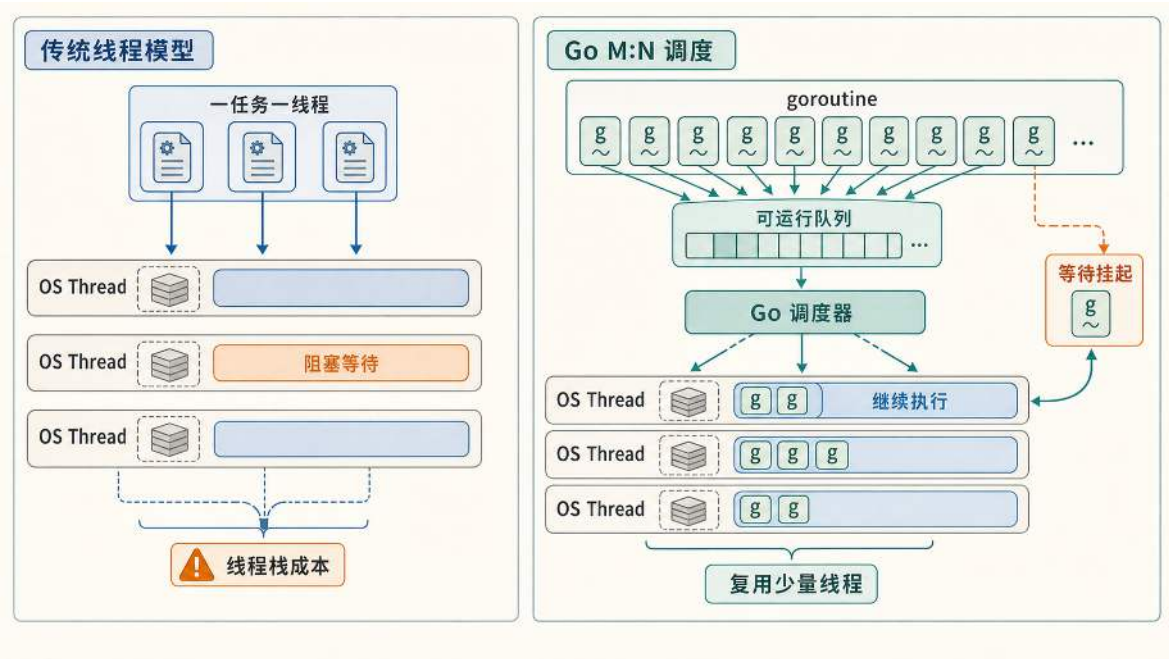


图 21: 传统线程模型让每个任务对应一条操作系统线程；Go 的 M:N 调度把大量 goroutine 复用到较少线程上，使等待型路径减少对操作系统线程的长期占用。

图21把这一资源收益对照成两种形态：在传统线程模型中，每个等待都背上一个完整的线程栈，连接规模扩大时栈成本线性累积；在 Go 的 M:N 调度中，大量 goroutine 由调度器分配到 OS 线程上执行，goroutine 挂起后，原线程可以继续承载其他可运行任务，使线程数量不再与等待任务数一一对应。

goroutine 也不会自动消除所有并发成本。复杂规则计算、大批量数据过滤或一次过大的消息编码属于计算型任务，它们会持续占用 CPU。如果任务彼此独立、计算量足够大，可以根据 CPU 核数安排有界并行；如果任务不可拆分、粒度过细，或者 goroutine 数量远超可用核心，继续拆分主要会增加调度和内存管理压力。因此，在线服务使用 goroutine 的判断标准应回到职责分工：等待型路径适合并发拆分，计算型路径需要控制并行度，权威状态提交则需要明确的串行化规则。对固定节拍的实时状态服务，这个规则可以是单写者主事件循环。所谓单写者（single writer），是指同一份权威状态只由一条固定执行路径提交修改，其他 goroutine 只能向这条路径投递消息；对普通请求服务，也可以使用按对象分区的队列、互斥锁或数据库事务规定提交顺序。后台 goroutine 完成工作后，应把结果交回相应的状态提交路径，而不是绕过既定同步规则直接改写共享状态。

两个代码块比较等待时间的叠加方式。假设服务器需要分别为 3 名玩家读取背包数据，主要耗时来自数据库或缓存返回结果。第一个版本按顺序读取三名玩家的背包，后来的玩家必须等待前一个查询完成；第二个版本用 goroutine 同时发起三次读取，让三段等待在时间上重叠。

顺序读取背包数据：等待串行叠加

```
1 func handlePlayer(playerID string) {
2     fmt.Printf("处理玩家 %s\n", playerID)
3     time.Sleep(2 * time.Second) // 模拟读取背包数据等 I/O 操作
4 }
5
6 func main() {
7     handlePlayer("A")
8     handlePlayer("B")
9     handlePlayer("C")
10    // 三次等待串行叠加，总耗时接近三次等待之和
11 }
```

通过 go 启动 goroutine 后，三个背包读取任务可以并发发起，等待时间更接近一次外部查询。这种写法改变的是等待的排列方式：单次数据库或缓存访问的耗时没有缩短，后来的玩家却不必排在前一个玩家的 I/O 等待之后。

Goroutine 并发读取背包数据：等待相互重叠

```
1 func main() {
2     var wg sync.WaitGroup
3     players := []string{"A", "B", "C"}
4
5     for _, player := range players {
6         wg.Add(1)
7         go func(id string) {
8             defer wg.Done()
9             handlePlayer(id)
10        }(player)
11    }
12
13    wg.Wait()
14    // 三次等待并发重叠，总耗时更接近一次等待时间
15 }
```

这个对比强调的是等待叠加方式的变化：顺序执行会把三次 I/O 等待串起来，并发执行则让这些等待在时间上重叠。对在线服务来说，这意味着某次数据库查询不必挡住其他连接的读写或消息推送。并发减少的是等待传播对整体吞吐的拖累，同时也会带来调度、内存和生命周期管理成本。

对于本节采用固定节拍和单写者模型的服务，goroutine 让连接读写、外部 I/O 和推送可以离开主事件循环单独等待，主循环仍然集中提交用户会话、订单状态和资源配置。这种分工把慢

I/O 和慢连接从权威状态路径中隔离出来，也让服务能够同时维持更多连接。执行流被拆开以后，问题不再只是“能不能并发”，而是这些等待如何被稳定地发现、分派和收束：连接什么时候可读，推送什么时候可写，后台查询什么时候返回，连接什么时候超时或关闭，都不能靠各条路径盲目轮询。这些等待点可以统一看作事件。服务器在事件发生时推进对应路径，在没有事件时让执行流退出等待或挂起等待，避免把整条主循环拖在同一个阻塞点上。

3.1.4 事件驱动与 I/O 多路复用：管理大量连接与等待

I/O 多路复用：把等待集中到内核 在线服务的事件驱动模型，可以从一条客户端连接开始理解。一条连接在大多数时间里没有新数据；只有当客户端发来请求、命令或心跳消息使连接变得可读，发送缓冲区允许继续写入，连接关闭，或者底层出现错误时，服务器才需要处理它。如果服务器反复检查每一条连接，空闲连接越多，无效检查越多；如果每条连接长期占用一条操作系统线程，线程栈和调度成本又会随着连接数增长。事件驱动把这个问题改写为一条稳定的程序组织方式：先等待哪些事情已经发生，再把事件分发给对应处理逻辑，处理完后继续等待下一批事件。客户端连接可读时，服务器读取并解析输入；连接可写时，服务器发送排队的推送结果；连接关闭或出错时，服务器清理会话、缓冲区和用户在线状态。

在操作系统看来，一条网络连接对应一个可被程序引用的句柄。服务器既不能为每条空闲连接长期保留一条线程，也不能反复扫描所有连接；前一种方式会放大线程栈和调度成本，后一种方式会把时间浪费在大量尚未就绪的连接上。

I/O 多路复用机制把等待集中到内核：应用登记一组关心的连接，只处理已经出现可读、可写、关闭或错误事件的句柄。Linux 中常见的 `select` 和 `epoll` 都解决“哪些连接已经就绪”这一问题；`epoll` 通过预先登记关注对象并返回就绪事件，更适合大量连接中只有少数连接活跃的场景。⁹

内核发现事件后，程序还要把不同事件交给相应处理逻辑。事件循环反复执行“等待事件、取出结果、分派处理”的结构，通常称为 Reactor 模式。¹⁰在 Go 服务中，这部分底层工作主要由运行时的 `netpoller` 承担：连接可读时，等待网络输入的 `goroutine` 被重新放回可运行队列，随后完成读取、分帧和解码。业务代码无需自行实现完整事件循环，但仍要把解析出的输入交给后续处理路径，而不能直接在网络读取路径中修改世界状态。

在本节采用的单写者实时服务中，这条客户端输入消息还不能直接修改业务状态。这些输入最终仍要回到按固定节拍推进的主事件循环，由它统一更新用户会话、资源配额和配置状态。如果读取路径绕过既定的状态提交规则，多个连接读路径就可能同时写入同一份状态，把前面已经隔离开的等待问题重新变成竞态问题。因此，读取路径和消费路径之间需要一个明确的数据交接点：读取路径负责产出客户端输入消息，消费 `goroutine` 或主循环负责按顺序取出消息并推进后续处理。

⁹Linux `epoll` 的接口语义见 Linux man-pages: <https://man7.org/linux/man-pages/man7/epoll.7.html>。

¹⁰Reactor 模式的经典描述见 Douglas C. Schmidt, “Reactor: An Object Behavioral Pattern for Demultiplexing and Dispatching Handles for Synchronous Events”, <https://www.dre.vanderbilt.edu/~schmidt/PDF/reactor-siemens.pdf>。

channel：读取路径与消费路径的交接点 最直接的实现方式是让读取路径把消息写入共享队列，再由消费路径从队列取出。但共享队列不仅要保存消息，还要处理并发访问、队列为空时如何等待、队列有新消息时如何唤醒消费者、队列满时如何让发送方感知压力。Go 语言里的 `channel` 把这几件事封装成语言级通道：发送方把消息放入通道，接收方按自己的节奏取出消息；如果一侧暂时没有准备好，运行时会记录等待关系，并在条件满足时唤醒对应 `goroutine`。

在在线服务中，`channel` 提供的是一个明确的消息交接点。读取路径把客户端输入放入通道，消费路径再按顺序取出并处理；业务状态的修改仍集中在后续消费路径中完成。这种组织方式体现了 Go 常见的并发思路：让数据以消息形式在 `goroutine` 之间移动，业务状态的读写则集中在少数明确的处理路径中。

`channel` 分为无缓冲与有缓冲两类。无缓冲通道更像一次同步握手，发送方和接收方必须同时就绪，消息才能完成交接；有缓冲通道则带有一段队列空间，发送方可以在缓冲未满时先把消息放进去，让接收方按自己的节奏消费。对在线服务而言，有缓冲通道常被用作连接或服务级输入队列，用来暂存客户端输入的短时突发；队列容量有限，持续过载最终会让发送方停在投递位置。

从并发语义上看，有缓冲的 `channel` 可以抽象为一段有限容量的缓冲队列，以及发送方和接收方的等待关系。当发送方投递消息时，若缓冲未满则可以入队；若缓冲已满且没有接收方完成交接，发送方会等待，直到通道重新具备发送条件。对称地，接收方在缓冲为空且没有发送方完成交接时也会等待。具体运行时可以使用锁、等待队列等数据结构实现这些行为，但这些内部结构不是 Go 语言向程序提供的接口保证。对应用设计而言，重要的是通道把发送、接收、等待和唤醒组织成一条具有明确阻塞条件的同步路径。

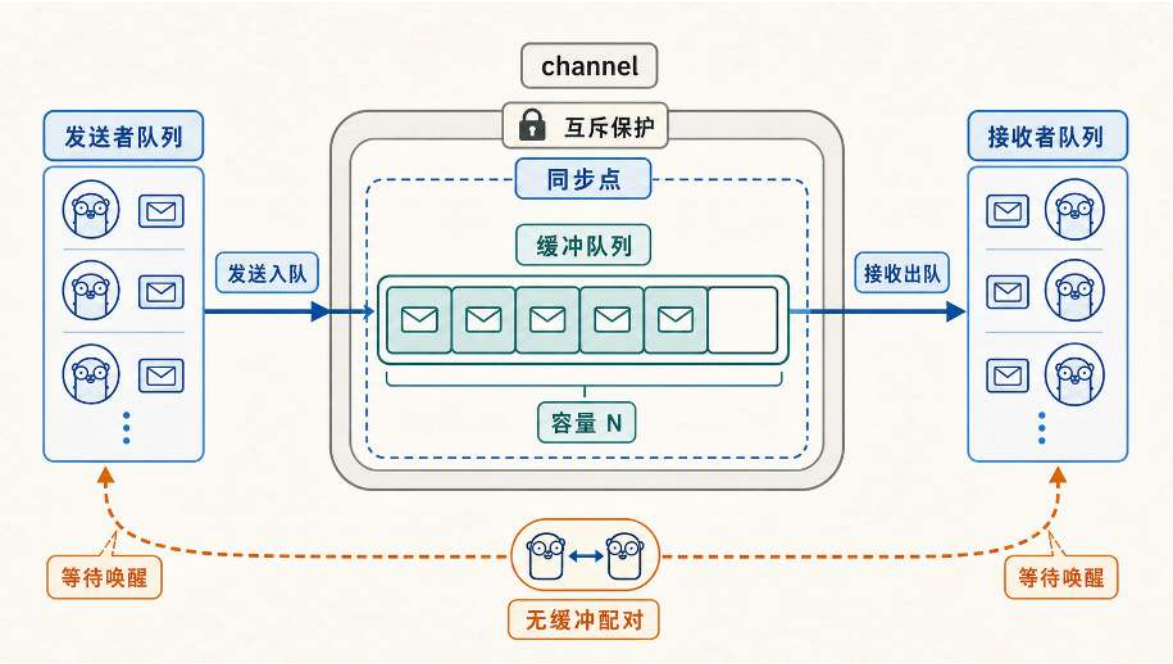


图 22: 从并发语义看，`channel` 用有限缓冲和两侧等待关系，把数据传递组织成具有明确阻塞条件的同步路径。

在一个采用服务级单写者循环的连接处理结构中，每个客户端连接至少会有读取路径和写入路径。读取 goroutine 负责从连接中读取字节流，例如从 WebSocket 连接读取客户端消息，解析成结构化消息后投递到有缓冲的 `channel`；服务级主事件循环再从输入队列中按节奏消费消息并集中裁决。这样，网络读路径只负责把输入变成消息并交出去，业务路径再按自己的节奏消费消息。当某条连接在弱网环境下连续发送消息，或者某次读取短暂变慢时，影响首先停留在这条连接的读取路径和输入队列中；主循环仍然可以按固定节拍消费已经到达的输入，不必因为单条连接的等待而暂停整个服务的推进。

连接读取路径：把消息交给服务级输入队列

```
1 type PlayerMessage struct {
2     PlayerID string
3     Action   string
4     Payload  []byte
5 }
6
7 func handlePlayerConnection(
8     playerID string,
9     svcInput chan<- PlayerMessage,
10 ) {
11     // 网络读循环：只负责读取、解析和投递（示意）
12     for {
13         msg, ok := readFromNetwork(playerID)
14         if !ok {
15             break
16         }
17         svcInput <- msg // 队列满时会阻塞，把压力传回读取路径
18     }
19 }
```

这段代码只展示输入交接路径。每条连接的读取 goroutine 只做 I/O、分帧、解析和轻量校验，再把消息投递到当前服务实例共享的输入队列；服务级主事件循环从同一个队列中按节奏消费消息并集中修改业务状态。单条连接退出时不能关闭这条共享队列，因为同一服务实例中的其他连接仍可能继续投递。

缓冲队列能够吸收短时间的输入突发，但容量有限。当客户端连续发送请求，而业务 worker 又因为状态计算、数据库写入或其他慢 I/O 处理不过来时，消息会先在 `channel` 中排队。如果这种差距只是短暂出现，队列会在后续消费中逐渐恢复；如果输入速度长期高于消费速度，队列就会被填满。

队列满后，读取 goroutine 再向 `channel` 投递消息时会停住。它停住以后，就不会继续从 WebSocket 连接中读取新的客户端输入。这样，下游业务处理能力不足的问题，会沿着 `channel`

反向传回网络读取路径。这种让上游感知下游处理能力不足、从而放慢输入推进的机制，就是背压。

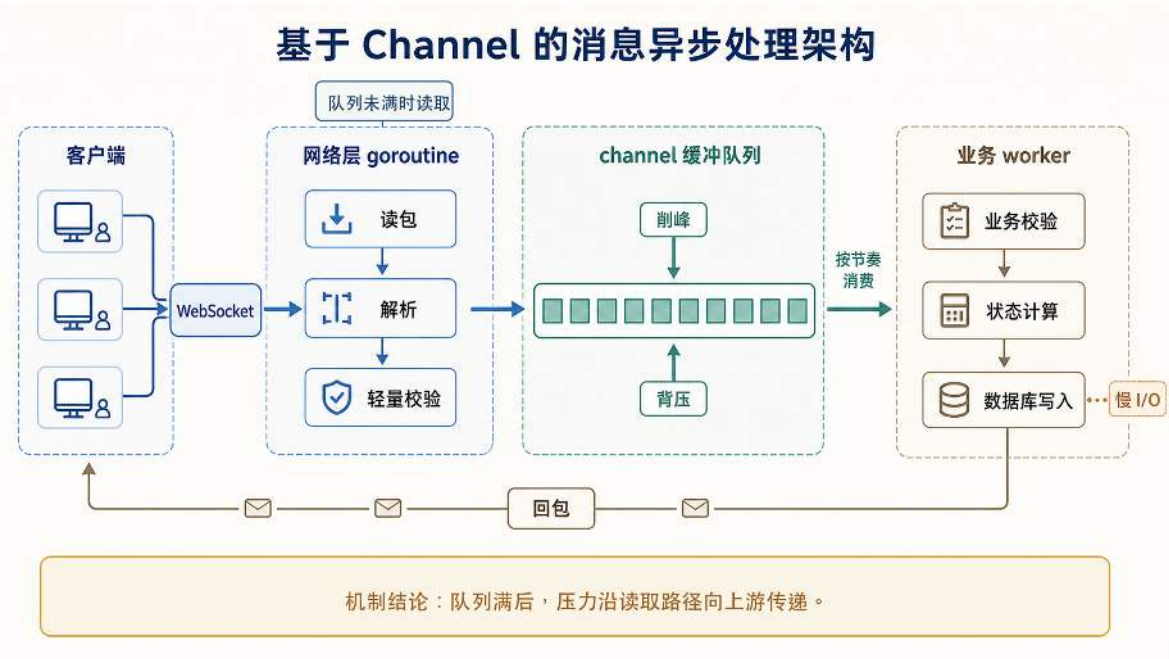


图 23: WebSocket 入口与业务处理通过 channel 解耦，缓冲队列满时压力会回到读取路径。

图23把这条压力传递路径放回在线服务的连接处理流程中。客户端消息先进入 WebSocket 连接，网络层 goroutine 负责读包、解析和轻量校验；解析后的消息进入 channel 缓冲队列；后续消费路径再按自己的节奏完成业务校验、状态计算和必要的外部访问。对采用单写者主循环的实时服务而言，需要修改权威业务状态的计算仍应回到主事件循环，数据库写入等慢 I/O 则应移出关键推进路径。消费路径中的慢 I/O 是一个具体限制：消费速度并不总是稳定的，一次数据库写入或外部访问变慢，就可能让队列开始积压。队列未满时读取路径可以继续投递；队列满后，读取 goroutine 会停在投递位置，压力沿读取路径逐步传回上游。

channel 在连接处理中的职责可以概括为三点：把客户端输入从网络读取路径交给业务处理路径，用有限缓冲吸收短时突发，并在下游处理不过来时把压力暴露回读取路径。

select：把多种等待写成可见分支 但连接处理路径并不只等待客户端消息。它还可能等待发送队列出现推送结果、等待心跳超时、等待连接关闭，或者等待上层取消信号。如果代码只停在某一个通道上等待，其他条件就可能被延迟处理。Go 的 select 用来把这些等待条件放在同一个控制结构中：只要某个分支已经就绪，它就可能被选中执行；如果多个分支同时就绪，select 不保证按源码中的先后顺序选择。因此，不能依靠 case 的书写顺序表达优先级；确实需要优先级时，应通过独立队列或分阶段选择显式实现。这样，等待不再隐藏在某一次阻塞调用里，而是被写成可见的分支：消息到达就处理，通道关闭就退出，超时到达就断开连接，上层取消就回收 goroutine。

超时分支：让等待拥有退出路径

```
1 func handleWithTimeout(playerID string, msgChan chan PlayerMessage) {
2     timer := time.NewTimer(30 * time.Second)
3     defer timer.Stop()
4
5     for {
6         select {
7             case msg, ok := <-msgChan:
8                 if !ok {
9                     return
10                }
11                timer.Reset(30 * time.Second)
12                processMessage(msg)
13            case <-timer.C:
14                fmt.Printf("玩家 %s 超时，断开连接\n", playerID)
15                return
16        }
17    }
18 }
```

这段代码采用 Go 1.23 及以上版本的通道式定时器语义：当主模块的 `go.mod` 声明 `go 1.23` 或更高版本时，`Reset` 返回后不会再收到旧配置产生的过期值，因此可以直接重置定时器。旧版本或显式启用旧定时器语义的程序仍需先停止并排空通道，不能把两种写法混用。¹¹

在这个示例中，`select` 同时等待消息通道和超时定时器。有消息就处理，通道关闭就退出，长期无消息就触发超时逻辑。类似结构常用于心跳检测与连接保活，避免僵尸连接长期占用资源。每一类等待都应当有可解释的退出路径；真实工程中常直接用 `context` 来统一取消链路。

同步与异步、阻塞与非阻塞 操作系统用 `epoll` 等机制等待连接句柄变得可读或可写；Go 运行时用 `netpoller` 把这些底层事件转换为 `goroutine` 的挂起与唤醒；业务代码用 `channel` 交接客户端输入和推送结果，再用 `select` 同时面对消息、关闭、超时和取消。如果不区分这些等待的语义，就容易把“结果怎样返回”和“当前执行流是否停住”混为一谈。

同步与异步描述结果沿什么路径返回：同步调用沿当前调用路径取得结果，异步调用则由稍后的事件、回调或结果句柄交付结果。阻塞与非阻塞描述当前执行流是否停在等待位置：阻塞调用会等待条件满足，非阻塞调用在暂时没有结果时立即返回。图24把同步/异步与阻塞/非阻塞拆成四种组合：异步接口也可能被调用方再次阻塞等待，同步接口也可能用立即返回的方式表达“暂时没有结果”。

¹¹Go 1.23 定时器通道语义说明见 <https://go.dev/wiki/Go123Timer>。

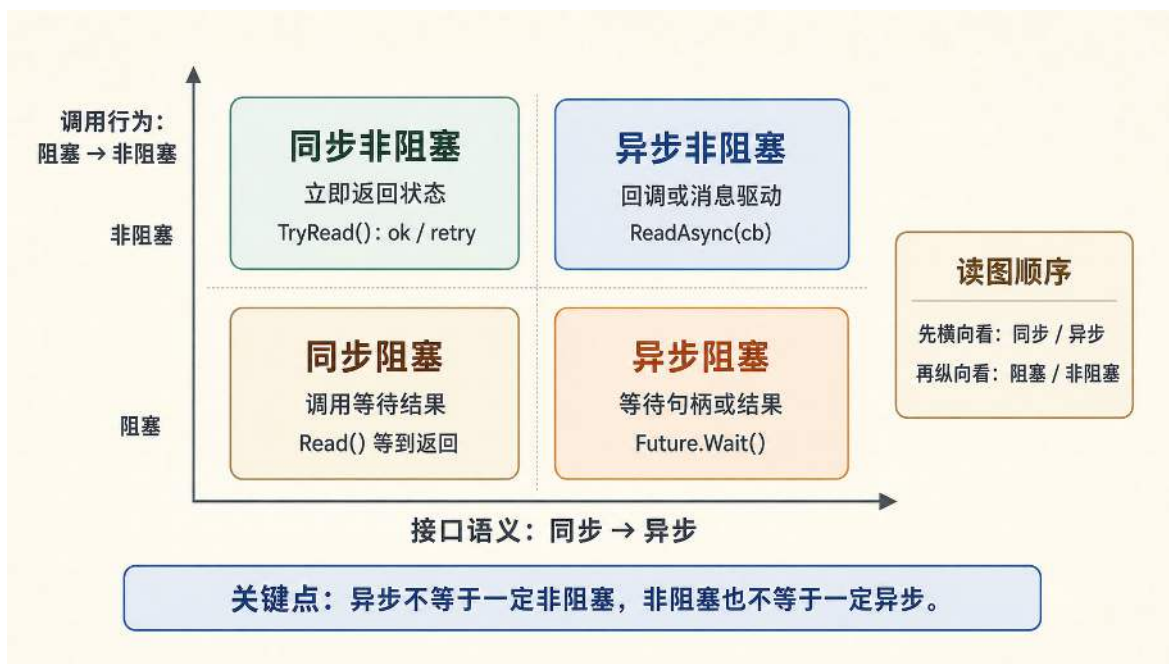


图 24: 同步/异步描述结果如何返回，阻塞/非阻塞描述当前执行流是否停住。

把这四种组合放回在线服务，可以得到更具体的判断。同步阻塞适合连接 goroutine 自己等待输入，但不能放进负责固定节拍状态推进的主事件循环，否则一个慢连接会拖住整个服务。同步非阻塞会在暂时没有数据时立即返回，但程序必须安排下一次检查。异步接口可以先提交数据库写入或跨实例查询；如果调用方随后立即等待结果，它仍然表现为异步发起、阻塞等待。事件驱动的网络路径则把空闲连接交给 netpoller 等机制，连接真正可读时再唤醒对应 goroutine，使服务把执行时间留给已经发生的输入事件和状态推进。

事件驱动并不表示在线服务里没有等待。客户端消息仍要等待网络到达，数据库写入仍可能变慢，空闲连接仍要等待心跳超时。它改变的是等待的组织位置：空闲连接交给运行时或事件循环等待，客户端输入到达后再唤醒读取路径；业务代码用 `select` 把消息、超时、关闭和取消写成明确分支。这样，某条连接暂时没有输入、某次外部 I/O 变慢，或某条连接已经断开，都不必把整条主循环拖在同一个等待点上。

goroutine 生命周期：取消、超时与容量限制 前面已经把连接读写、业务处理、慢 I/O 等工作拆到不同 goroutine 中。这种拆分提高了并发能力，也带来一个新的管理问题：这些 goroutine 什么时候应该结束。客户端断开连接后，读取 goroutine 应该退出；一次数据库写入超过请求期限后，等待它的业务路径不应继续占住资源；服务器准备关闭某个服务实例时，相关的连接处理、后台落库和推送任务也应尽快停止。如果没有统一的退出信号，后台 goroutine 可能继续等待数据库、计时器或外部接口，最终变成难以定位的 goroutine 泄漏。

在 Go 服务端中，这类退出信号通常由 `context.Context` 承担。一次用户请求、一次数据库访问、一个连接会话，都可以带着同一个 `context` 向下传递。当连接断开、请求超时或服务准备关闭时，上层取消这个 `context`，下游 goroutine 就能通过 `<-ctx.Done()` 收到退出信号。把

context 放进 select 分支后，业务代码就能把“有消息就处理、超时就放弃、父任务取消就退出”写成同一套控制结构。

除了管理已经启动的 goroutine，在线服务还要限制同时进入某条路径的任务数量。例如同一个时刻不能让过多订单结算同时写数据库，也不能让所有用户请求一起挤进某个昂贵的匹配或存档路径。如果只负责启动 goroutine，却不给这些路径设置容量上限，高压下队列会继续膨胀，超时也会被拖到更晚才暴露。

带缓冲的 channel 可以用来表达这种容量上限。它可以看作一个容量为 N 的信号量：往通道里放入一个空结构体表示占用一个名额，取出一个空结构体表示归还名额；当前元素数是已占用名额，剩余容量才是还能进入的并发数。当获取名额的动作也写进 select 时，程序就能同时表达“拿到名额就执行”和“等太久就放弃或降级”，避免高压下无休止排队。

容量令牌：把并发上限和超时写进同一条路径

```
1 var sem = make(chan struct{}, 100) // 容量=100, 最多允许100个并发
2
3 func withLimit(timeout time.Duration, fn func()) bool {
4     select {
5         case sem <- struct{}{}: // 获取令牌
6             defer func() { <-sem }() // 归还令牌
7             fn()
8             return true
9         case <-time.After(timeout):
10            return false
11     }
12 }
```

这些机制共同服务于同一个目标：让连接等待、业务等待和资源等待都有明确位置，而不把所有不确定性都压回同一条关键处理路径。

3.1.5 请求处理链：串联进程、线程、协程与异步通信

前面已经分别讨论了进程与线程、goroutine、channel、select、context 和容量令牌。这些机制需要放进一条完整的请求处理链，才能看清各自承担的职责：连接由谁等待，输入如何交给业务处理，慢 I/O 如何移出主路径，关键状态又由谁按确定顺序更新。在线服务的一轮事件处理可以把这些职责串成一条完整路径。

以固定节拍推进业务状态的服务为例，假设每 33ms 做一次状态更新与结果推送。与此同时，客户端的输入是连续到来的，有的来自光纤宽带，有的来自移动网络，延迟与抖动差异很大。系统的关键矛盾在于避免把整轮推进节奏绑在最慢的那段等待上，而不只是把某个函数算得更快。理解这一点后，进程、线程、协程与异步通信就不再是抽象名词，而是用来切断等待传播、控制故障半径与管理并发规模的工具。

进程隔离：让故障半径可控 把一类业务放在一个独立进程里，是一种常见的工程选择。它的意义首先是隔离：业务进程崩溃时，影响被限制在该实例范围内，其他服务仍可继续运行；其次是资源治理：每个进程都有独立的内存空间和文件句柄表，更容易做部署、监控与自动拉起。它的代价也很明确：跨实例交互需要跨进程通信，一旦用户要跨实例移动，状态交接就不再是一次函数调用，而会变成一次协议与一致性问题。这类跨进程协作会在第 4.3.4 节继续展开，本章先把视角收敛在单机进程内的并发组织方式上。

线程与多核：计算段最终要落在 CPU 核上 无论上层抽象多么轻量，真正执行指令的仍是操作系统线程。在线服务的计算段，例如数据过滤、规则计算、批量推送的序列化，最终都会被调度到若干操作系统线程上，由多个 CPU 核执行。并行在这里发挥作用：当瓶颈来自算力不足时，利用多核可以压缩单轮计算时间。与此同时，线程也暴露出一个具体限制：阻塞不会消失，只会换一种位置出现。若把网络读写或磁盘 I/O 直接放在少量关键线程上，阻塞仍可能把推进节奏拖进队列。

协程与连接：让等待只阻塞当前工作线 这就引出了协程抽象的价值。对在线服务而言，最自然的拆分方式是以连接为单位拆执行流。每条连接的网络读写可能长时间等待，但这种等待应只影响这条连接，而不是把整个服务拖慢。goroutine 这样的轻量执行单元，使得每条连接都可以有独立的读写工作线：慢连接在自己的 goroutine 里等待，不会阻塞其他连接的处理；连接断开时也能自然回收与退出。与此同时，协程的轻量并不意味着可以无限制创建，系统仍需要在 goroutine 数量、队列长度与资源占用之间保持可观测与可控。

异步通信：把数据所有权与节奏写进程序 当连接级 goroutine 与主事件循环的执行流被拆开后，还需要一种方式把输入汇入主循环，又把主循环的结果分发回各个连接。channel 在这里扮演的是受同步保护的队列，它让数据以消息的形式流动，并把背压点显式化。输入通道的长度、输出通道的长度、以及在高压下是否允许丢弃或降级，都会直接决定系统在尾延迟上的表现。select 则把超时、取消与退路写进控制流，使等待从隐式变成可推理的结构。

在线服务的一种并发骨架（示意）

```
1  type Event struct {
2      ConnID int
3      Msg     Message
4  }
5
6  type Client struct {
7      conn net.Conn
8      out  chan Diff // 每条连接自己的有界发送队列
9  }
10
11 func runServer() {
12     in := make(chan Event, 8192)          // 输入事件队列：吸收网络抖动
13     persist := make(chan Persist, 1024) // 异步写回队列：把慢 I/O 移出主路径
14     clients := newClientRegistry()
15
16     go persistLoop(persist)
17     go mainLoop(in, persist, clients) // 集中更新业务状态
18
19     for { // 接入循环（示意）
20         c := acceptConn()
21         client := &Client{conn: c, out: make(chan Diff, 256)}
22         clients.Add(client)
23         go readLoop(c, in) // 阻塞读只影响该连接
24         go writeLoop(c, client.out) // 慢写停留在该连接
25     }
26 }
27
28 func mainLoop(
29     in <-chan Event,
30     persist chan<- Persist,
31     clients *ClientRegistry,
32 ) {
33     ticker := time.NewTicker(33 * time.Millisecond)
34     defer ticker.Stop()
35
36     const maxEventsPerTick = 2048
37     for range ticker.C {
38         // 每轮只在预算内消费输入，避免事件洪峰拖垮推进节奏
39         drainInput:
40         for i := 0; i < maxEventsPerTick; i++ {
41             select {
42                 case e, ok := <-in:
```

这段骨架把前面的概念对应到在线服务的具体运行路径上：进程划出服务实例的故障影响范围，线程承载实际执行，goroutine 分别负责连接读写、持久化和主循环，channel 把客户端输入与异步写回任务交给后续路径，select 让主循环在每轮预算内消费已经到达的事件。这里的主循环让业务状态主要由同一条循环集中修改：输入事件先进入队列，再按每轮预算被消费，最后统一推进业务状态并推送增量。代码中的 `queueDiffs()` 体现了状态同步的两个关键设计：主循环每轮只按订阅关系生成发生变化的部分，并把结果放入各连接的有界发送队列；独立写流程再完成可能阻塞的 socket 写入，不让慢客户端停住主循环。每条连接的 out 队列容量为 256，队列满时的处理策略沿用第 2.4.2 节“读写分离”中已经建立的分类原则：可覆盖的实时状态（如位置、朝向）合并为最新版本，避免同一对象反复排队；关键消息（如交易结果、奖励发放）不得静默丢弃，必要时通过超时、重同步或断开异常连接来暴露故障；主循环本身不能阻塞在发送队列上，否则整轮推进都会被最慢的客户端拖住。这种增量同步策略直接决定了推送的数据量：如果 100 名客户端同时在线，但本轮只有 3 个状态发生变化，增量同步只需要把这 3 个变化按可见关系发送给相关客户端，而不是把 100 个完整状态发送给所有人。

不过，把业务状态主要集中到 mainLoop 并不能覆盖所有共享数据。真实服务里，连接状态、用户会话、缓存视图、跨对象交易、异步持久化结果，都可能被不同 goroutine 接触；不受约束的实现还可能绕过输入队列，直接读写同一份对象。

问题会出现在“判断”和“修改”之间。以电商库存扣减为例，两个订单几乎同时扣减同一件商品时，两个执行流都可能先读到“库存仍然大于零”，随后分别扣减并确认订单。从每条执行流内部看，代码都像是按顺序执行的；放到并发环境里，库存却可能变成负数。这类错误不会表现为程序立即崩溃，而是表现为业务规则被破坏：同一份库存可能被超卖，或者不同客户端看到互相矛盾的订单状态。以游戏为例，两名玩家同时拾取地图上唯一的宝箱，也会出现同类问题。

这就把并发问题从“怎样让更多任务同时推进”推进到另一个层面：当多个执行流同时接触关键状态时，服务器怎样保证规则仍然只有一个确定结果。

3.2 并发正确性：保护共享状态

上一节把等待路径拆成了多个并发执行流，使更多请求能够同时推进。当这些执行流同时读取或修改共享状态时，系统还必须保证订单库存、账户余额、协作文档版本或游戏奖励仍然只有符合规则的结果。吞吐提升不能以破坏状态正确性为代价，因此并发结构必须配套明确的同步规则。

以电商库存扣减为例，两条处理路径几乎同时扣减同一件商品。路径 A 检查到“库存仍大于零”，还没来得及写回扣减后的数量；路径 B 也读到旧库存，于是两件商品被同时卖出，总库存变成了负数。以游戏 NPC 奖励为例，两条路径几乎同时触发同一只 NPC 的奖励发放，也可能出现同一份奖励被发出两次的情况。这类竞态暴露了并发环境下规则确定的三层隐患。第一，原子性：检查与修改必须不可被插队，否则先到的规则失效。第二，可见性：在多核环境下，路径 A 的写入可能停留在本地缓存中，尚未以其他核心可见的方式发布出去；路径 B 即使在时间上稍后读取，也未必能看到已更新的值。第三，有序性：编译器和 CPU 可能在不改变单线程

结果的前提下重排读写，使得代码中的先后顺序在并发下不再可靠。

并发程序需要用同步机制把关键动作之间的先后关系显式地写进程序：互斥锁让“检查库存”和“写回扣减后数量”在同一段受保护的代码里完成；channel 让发送方交出数据后接收方再继续处理；条件变量让等待者在条件变化后被通知并重新检查条件。这些机制形式不同，但都在回答同一个问题：哪个动作已经完成，哪个动作可以基于它继续。在并发语义中，这类可依赖的先后关系称为 *happens-before*（先发生于）。¹²

把这个场景落到代码层面。goroutine A 将共享计数从正数改成扣减后的值，表示库存已被占用。如果这次写入没有通过锁释放、channel 发送等同步事件传递给另一条处理路径，goroutine B 仍可能按旧状态继续执行。问题在于共享状态没有被放进受保护的使用过程里：检查条件是否满足、写回更新后的状态、基于结果继续处理，这几步必须由同一种同步关系串起来。以互斥锁为例，某条处理路径持有锁时，其他处理路径只能等待；等它完成检查和写回并释放锁之后，下一条处理路径才能进入同一段代码，并基于已经更新后的状态继续判断。并发程序里许多看似偶发的错误，本质上都是可见性与有序性没有被写进程序。要让程序获得一个可依赖的顺序（即前一个动作已经完成，后一个动作可以基于它继续），必须由某条同步事件在两个动作之间建立关系。

图25对照了互斥锁与 channel 两种建立这种关系的常见路径。

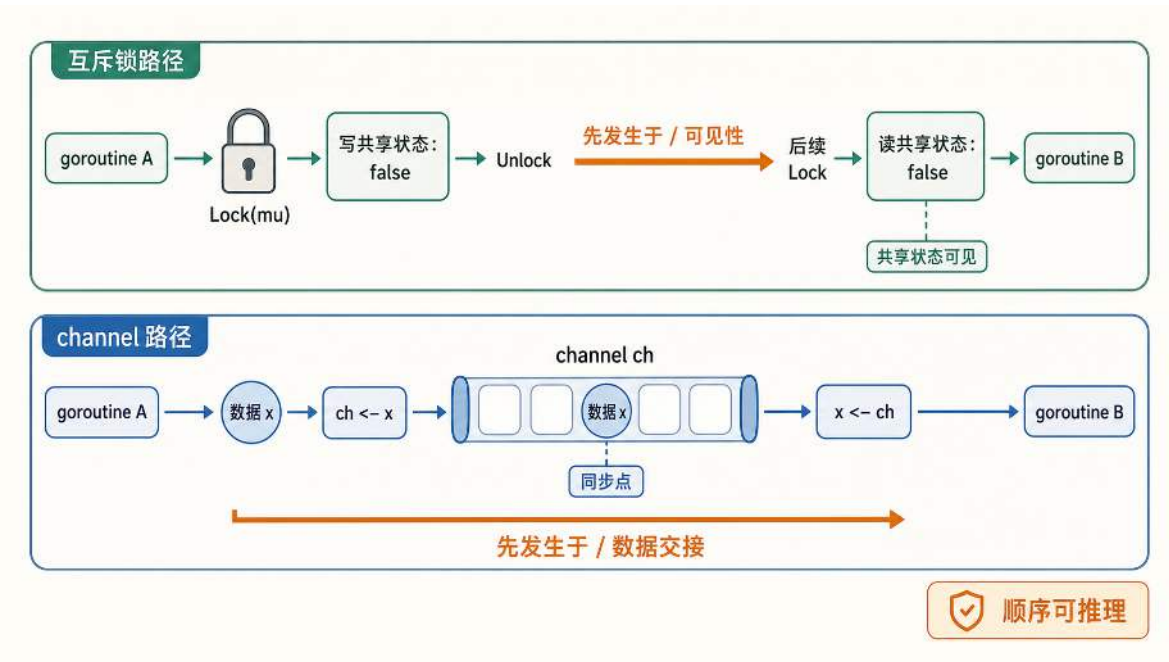


图 25: 互斥锁通过 Unlock 与后续 Lock 建立可见性，channel 通过发送与接收的交接点建立同步关系，两者都在 goroutine 之间形成可推理的先后顺序。

在互斥锁路径中，A 在持有锁期间写共享状态并执行 Unlock，B 后续成功 Lock 之后再读取共享状态，Unlock 与后续 Lock 之间建立了可见性；在 channel 路径中，A 把数据发送到

¹²Go 语言对 happens-before、同步事件和内存可见性的正式定义见官方文档 The Go Memory Model: <https://go.dev/ref/mem>。

channel, B 从同一个 channel 接收后才继续执行, 发送与接收的交接点就是同步点。两条路径形式不同, 但都让前一个动作的结果对后一个动作可见。

围绕 NPC 奖励归属, 后文按三步展开这一组同步问题。第一步是从规则出发识别不变量与临界区, 并用最小反例看见竞态条件如何生成。第二步用互斥锁把临界区收敛成唯一顺序, 同时补上活性视角, 说明死锁与饥饿为什么会让系统走不动。第三步从忙等过渡到条件变量, 把等待-通知机制与 for 复检条件讲清楚, 让正确性不再依赖侥幸的调度时机。

3.2.1 共享资源竞争：临界区与竞态条件

并发程序最常见的约束之一, 是确保共享资源在同一时刻只被一个执行流修改。以在线游戏的 NPC 奖励为例: 地图上随机刷新一只 NPC, 玩家走近触发任务后, NPC 送出宝物并立刻消失, 同一个 NPC 只能被成功触发一次。

这条规则之所以值得单独拿出来, 是因为它代表了服务端最常见的一类约束: 对共享资源的唯一归属。库存扣减、余额转账、限量抢购、文档协同编辑的冲突解决, 本质上都在回答同一个问题: 当多个请求几乎同时到达时, 系统如何保证一份资源只被分配一次, 并且在所有用户看来是公平可解释的。如果这一点守不住, 问题不会停留在一次显示错误上, 它会迅速演化成可被利用的漏洞, 破坏经济系统或业务规则的可信度。

把这条规则写成状态转换会更清楚。NPC 是否还能送出宝物、库存是否仍大于零、账户余额是否足够, 都可以抽象为一个布尔状态或数值状态, 从满足条件到不满足条件只允许发生确定次数。顺序程序里, 这个转换天然被执行顺序保护; 并发程序里, 执行顺序变成了交错顺序, 必须显式地规定: 检查条件与写回状态是一件不可分割的事。竞态条件的出现, 正是因为这段状态转换的检查与写回没有被同步机制保护成不可分割的整体。

在顺序程序里, 这条规则几乎不需要额外说明; 但在并发程序里, 它会立刻变成一个同步问题。两个请求可能同时到达, 两个 goroutine 也可能几乎同时执行到同一段代码。如果这段代码同时读取并修改了同一份状态, 例如 NPC 是否已被触发、库存是否仍大于零, 那么它就是临界区。临界区缺乏保护时, 就会出现竞态条件: 两个执行流都通过了检查, 都认为自己有权修改状态, 最终产生两个成功的结果。

竞态条件的根源, 通常在于检查与修改之间存在时间窗口。把这段窗口收敛到不可分割的原子区, 是守住不变量的唯一可靠做法: 让“读取是否还能送、决定是否送出、写回已送出”这三步在同一段临界区内顺序完成, 期间不允许其他执行流插入。图26对照了未保护和受保护两种执行顺序: 未保护时两个 goroutine 先后读到旧状态、都通过检查、都进入写回前的间隙, 最终重复发放; 用互斥锁保护后, 检查与写回被收紧在同一段临界区内, 后到的请求只能看到前者更新后的状态, 因此只有一条路径能通过。

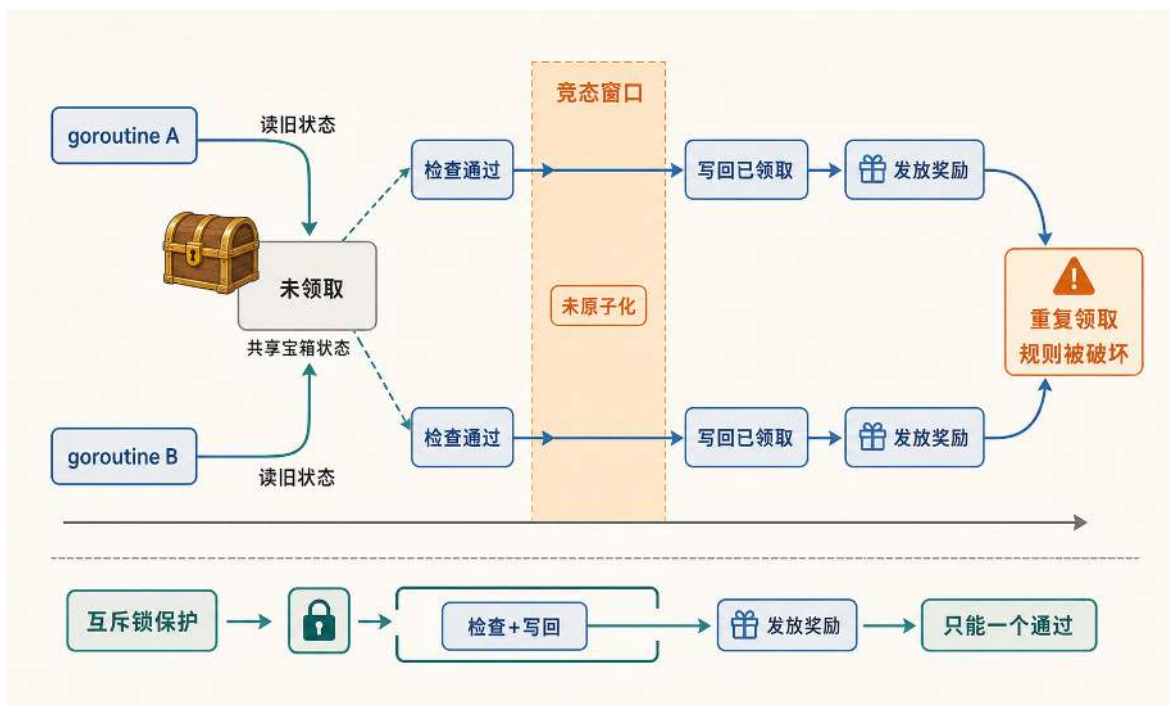


图 26: 竞态条件的时序分析: 缺乏同步保护导致两个 goroutine 同时通过检查并修改共享状态。

从设计视角看, 这段需要原子化的组合操作很像一笔小事务。它要么完整发生, 要么不发生, 中间状态不能被外界观察。互斥锁是一种实现这种小事务的方式, 事件队列和单写者也是。这里先用互斥锁建立基本概念, 因为它能把不可分割的要求准确翻译成运行时约束。

临界区通常不只是单独的那行赋值, 而是维持某个不变量的一段组合操作。对 NPC 奖励归属而言, 不变量是宝物只能送出一次。要维持这个不变量, 程序必须把读取是否还能送、决定是否送出、以及写回已送出这几步视为一个整体。只要其中任何一步可能被别的 goroutine 插入, 规则就可能被破坏。

更准确地说, 竞态并不要求两段代码真的在同一纳秒执行, 它只需要检查与修改之间存在未受保护的间隔。这个间隔可能来自网络延迟、磁盘 I/O、协程调度, 甚至只是一次看似无害的函数调用。并发程序最容易出错的地方, 正是把源代码里挨在一起的两步操作误当成执行时必然连在一起。

一段极简代码可以重现这个竞态窗口, 说明临界区缺乏保护时会发生什么。全局变量 `npcHasTreasure` 表示 NPC 是否仍然可以送出宝物, `greetNPC` 模拟玩家打招呼触发任务的流程: 先检查, 再做一点延迟, 然后修改状态并返回结果。

危险代码：缺乏同步保护的并发访问

```
1 var npcHasTreasure = true // NPC 是否还可以送出宝物（共享状态）
2
3 func greetNPC(playerID string) {
4     if npcHasTreasure { // 步骤1: 检查是否还能触发
5         time.Sleep(time.Millisecond) // 步骤2: 模拟延迟
6         npcHasTreasure = false // 步骤3: 标记已触发
7         fmt.Printf("玩家 %s 获得宝物\n", playerID)
8     } else {
9         fmt.Printf("玩家 %s 触发失败, NPC 已消失\n", playerID)
10    }
11 }
12
13 func main() {
14     var wg sync.WaitGroup
15     wg.Add(2)
16     go func() { defer wg.Done(); greetNPC("A") }()
17     go func() { defer wg.Done(); greetNPC("B") }()
18     wg.Wait()
19 }
```

多次运行这段代码，通常会出现两名玩家都获得宝物的结果。在真实系统里，类似的超卖或重复发放也会以同样方式发生。问题的关键在于检查与修改之间存在明显的时间窗口：执行流 A 检查到条件满足后进入短暂延迟，执行流 B 在这个窗口内也完成了检查，同样进入成功分支。即使延迟只有 1 毫秒，在高并发场景下这个窗口仍然足以触发错误。

这里的 `time.Sleep` 仅用于放大窗口以便观察。真实系统中不需要显式睡眠也会出现同类窗口：一次 JSON 反序列化、一次日志写入、一次缓存访问，都可能把执行流切走。并发规模越大，可能的交错排列越多，原本看似偶发的问题就会越来越频繁，直到在压力场景下近乎必现。

这暴露了一个根本矛盾：并发让多个请求可以同时推进，但某些规则天生要求串行化。系统既要保持整体吞吐，也要让关键的共享状态修改在同一时刻只能由一个执行流完成。要让这条规则在并发下仍然成立，需要互斥锁把临界区收敛成唯一顺序。

3.2.2 互斥锁：把临界区收敛成唯一顺序

临界区收敛与公平性 要解决竞态，核心是把检查与修改变成不可分割的原子区，在同一时刻只允许一个 goroutine 进入临界区执行，其他 goroutine 必须等待。互斥锁（Mutex）就是最常见的同步原语：进入临界区前加锁，离开时解锁。

从抽象上看，互斥锁做了两件事。第一，它把临界区变成一条单车道，同一时刻只允许一个执行流通过，从而保证检查与修改不会被插队。第二，它在通过的两端建立内存同步，让临界区

内的写入对随后进入的执行流可见，临界区内讨论顺序时也就有了可依赖的因果关系。这正是先发生于关系在工程里的落点。

在 Go 中，标准库提供了 `sync.Mutex`。它只有两个核心方法：`Lock()` 获取锁（不可用则等待），`Unlock()` 释放锁。为了避免因为提前返回而忘记解锁，工程上通常会用 `defer` 保证临界区结束时一定释放锁。

互斥锁可以直接修复前面的竞态。函数在检查与修改共享状态之前加锁，并用 `defer` 确保退出时一定解锁。这样，这段临界区在同一时刻只会被一个 goroutine 进入。

安全代码：使用 Mutex 保护共享资源

```
1 var npcHasTreasure = true
2 var npcLock sync.Mutex // 互斥锁
3
4 func greetNPCSafe(playerID string) {
5     npcLock.Lock() // 获取锁，进入临界区
6     defer npcLock.Unlock() // 延迟释放锁
7
8     if npcHasTreasure {
9         npcHasTreasure = false
10        fmt.Printf("玩家 %s 获得宝物\n", playerID)
11    } else {
12        fmt.Printf("玩家 %s 触发失败，NPC 已消失\n", playerID)
13    }
14 }
15
16 func main() {
17     var wg sync.WaitGroup
18     wg.Add(2)
19     go func() { defer wg.Done(); greetNPCSafe("A") }()
20     go func() { defer wg.Done(); greetNPCSafe("B") }()
21     wg.Wait()
22 }
```

此时运行结果稳定为只有一名玩家成功、另一名玩家失败。互斥锁把原本可能并发执行的临界区串行化，让共享状态的修改有了唯一顺序。这里使用 `defer`，是为了让正常返回和提前返回都执行 `Unlock()`，避免因遗漏解锁使其他 goroutine 永久等待。安全版同时删去了人为延迟，因为它不再承担复现竞态窗口的作用；即使保留该延迟，互斥锁仍能保证唯一结果，但真实代码不应在持锁期间执行慢操作。

不过，互斥锁保证的是同一时刻只有一个执行流进入临界区，它并不自动保证先到先得的顺序。谁先拿到锁，取决于调度与竞争时机。在很多规则里，只需要保证唯一性就已经足够；但如

果规则必须按到达顺序或时间戳裁决，就必须把顺序写进协议，例如把请求入队交由一个仲裁者按序处理，或者在消息上携带可验证的序列号。公平不是锁的副产品，它必须作为规则的一部分被明确设计。

活性陷阱：死锁与饥饿 使用互斥锁时有三点常见陷阱。第一，不要复制 `sync.Mutex`；通常把它作为结构体字段，或以指针共享同一把锁。第二，临界区要尽量小，只包住共享状态的读写，把纯计算放在锁外。第三，尽量避免在持有锁时再去获取另一把锁；如果确实需要，必须规定统一的加锁顺序，否则很容易死锁。第三点看似只是编码规范，背后指向的是活性：系统不仅要算对，还要能持续向前推进。活性失守时，用户看到的现象更直接：服务突然停住，或者某一类操作总是超时。死锁和饥饿是两类典型的活性陷阱。

死锁是最极端的一类活性问题。它并不要求代码里有明显的错误语句，只要两个执行流以不同顺序获取多把锁，就可能形成环等待。在最小死锁示例中，A 先拿 `lockA` 再拿 `lockB`，B 反过来。两条执行流各自拿到第一把锁后，再同时尝试获取对方持有的锁，便会形成确定的环等待。

从结构上看，死锁往往同时具备四个特征：资源互斥、占有并等待、不可抢占、环等待。互斥是锁的本性；占有并等待出现在拿着一把锁去等另一把锁；不可抢占意味着别人无法强制把锁收回；最后的环等待则由 A 等 B、B 等 A 构成循环。工程上最有效的切入点通常是破坏环等待，也就是统一加锁顺序。

死锁：锁顺序不一致导致环等待（示意）

```
1  var lockA sync.Mutex
2  var lockB sync.Mutex
3  var aLocked = make(chan struct{})
4  var bLocked = make(chan struct{})
5
6  func goroutineA() {
7      lockA.Lock()
8      defer lockA.Unlock()
9      close(aLocked) // 通知 B: A 已持有 lockA
10     <-bLocked      // 等待 B 持有 lockB
11
12     lockB.Lock()
13     defer lockB.Unlock()
14 }
15
16 func goroutineB() {
17     lockB.Lock()
18     defer lockB.Unlock()
19     close(bLocked) // 通知 A: B 已持有 lockB
20     <-aLocked      // 等待 A 持有 lockA
21
22     lockA.Lock()
23     defer lockA.Unlock()
24 }
25
26 func main() {
27     var wg sync.WaitGroup
28     wg.Add(2)
29
30     go func() {
31         defer wg.Done()
32         goroutineA()
33     }()
34     go func() {
35         defer wg.Done()
36         goroutineB()
37     }()
38
39     wg.Wait() // 两条路径互相等待，程序无法继续推进
40 }
```

示例中的 `aLocked` 和 `bLocked` 只用来确保两条路径都已持有第一把锁，从而稳定复现环等待；它们不是解决死锁的机制。

在在线服务里，死锁最常见的形态之一是双对象操作。比如转账需要同时修改双方账户余额，交易需要同时修改买卖双方库存与资金。如果两条路径各自按自己的顺序加锁，在并发对称触发时就会形成环等待。很多死锁来自对称结构在高并发下更容易被触发的锁顺序冲突，而不一定来自复杂业务逻辑。

运行最小死锁示例时，主 goroutine 等待 `WaitGroup`，而 A、B 又互相等待锁，Go runtime 通常会报出 `fatal error: all goroutines are asleep - deadlock!`。¹³这一现象只在最小示例中成立。真实服务里往往还有网络轮询、定时器或其他活跃 goroutine，局部路径的死锁通常不会被 runtime 直接识别为全局死锁，而是表现为部分请求永久挂起或持续超时。死锁的发现因此不能只依赖 runtime 的全局检测，更要靠锁顺序规约、超时机制和可观测指标来暴露。

工程上避免死锁的核心在于把资源获取变成可推理的协议。最常用的做法是规定统一的加锁顺序，例如总是按对象 ID 从小到大获取锁，任何跨对象操作都遵守同一顺序；其次是尽量减少同时持有多把锁的场景，把跨对象逻辑改写为消息串行化或分阶段提交，让临界区更窄、更短。

饥饿比死锁更隐蔽，也更贴近在线服务的真实体验。饥饿的含义是系统整体仍在运行，但某个执行流或某一类请求长期得不到资源，等待时间没有可依赖的上界。

在实时服务里，饥饿常表现为某类输入事件一直得不到处理。战斗事件持续推进，但移动事件的延迟越来越大，玩家会感觉操作像被丢弃。造成饥饿的原因有时并非算力不足，而是调度策略长期把某类请求放在队尾。

饥饿常见于两种原因：一种是锁或调度策略缺乏公平性，某些任务总被插队；另一种是人为引入了优先级，却忘了给低优先级留下最低保障。以下事件处理示例说明后一种情况：如果总是优先处理战斗事件，而战斗事件又持续不断，那么移动事件就可能一直排不到。

¹³Go runtime 的全局死锁检查逻辑可核对官方源码：<https://go.dev/src/runtime/proc.go>。

饥饿：优先级策略让某类事件长期得不到处理（示意）

```
1 type Event struct{ /* ... */ }
2
3 func loop(combat <-chan Event, movement <-chan Event) {
4     for {
5         select {
6             case e := <-combat:
7                 handleCombat(e)
8                 continue
9             default:
10            }
11            select {
12                case e := <-combat:
13                    handleCombat(e)
14                case e := <-movement:
15                    handleMovement(e)
16            }
17        }
18    }
```

要避免饥饿，通常需要为系统设置最低保障：例如采用配额策略，每处理 N 个高优先级事件就必须处理 1 个低优先级事件；或者把队列变成有界队列，并在拥塞时对高优先级也施加背压，让系统能在极端负载下保持可控。对实时服务而言，活性只保证系统能继续推进往往还不够，还必须把等待收敛到可接受的尾延迟范围内，否则用户感知到的就是尾延迟失控形式的不公平。

3.2.3 等待与通知：从忙等到条件变量

互斥锁解决了临界区的唯一顺序问题，但当条件不满足时，执行流仍需要一种高效等待的方式，而不是不断抢锁。以一个容量有限的任务队列为例：消费者可以从中取走任务，生产者也可以向其中放入新任务。如果队列空了，消费者可以选择等待最多一段时间；如果队列满了，生产者也可能希望等待到有空间再继续。

只用互斥锁很容易写出轮询：周期性加锁检查条件，不满足就解锁，等待一小段时间后再试；如果连这段等待也去掉，就会退化为持续占用 CPU 的忙等。两种写法都没有建立等待与状态变化之间的通知关系，只是轮询的 CPU 消耗通常低于持续忙等。

频繁轮询或忙等在高并发下会制造更坏的反馈：等待者反复抢锁，导致真正持锁的人也更难推进，锁竞争变重，尾延迟被进一步放大。对延迟敏感的在线服务来说，这种把等待变成负载的方式往往是最先需要剔除的。

互斥锁 + 定期轮询（反例示意）

```
1 type TaskQueue struct {
2     Items int
3     mu     sync.Mutex
4 }
5
6 func takeByPolling(q *TaskQueue, timeout time.Duration) bool {
7     deadline := time.Now().Add(timeout)
8     for time.Now().Before(deadline) {
9         q.mu.Lock()
10        if q.Items > 0 {
11            q.Items--
12            q.mu.Unlock()
13            return true
14        }
15        q.mu.Unlock()
16        time.Sleep(10 * time.Millisecond)
17    }
18    return false
19 }
```

用更长的 `Sleep` 或指数退避降低轮询频率，本质上仍是在用经验参数猜测等待时间：等待时间短了会频繁抢锁，时间长了又会错过本该及时响应的机会。轮询没有把等待和唤醒的因果关系写进程序；去掉 `Sleep` 后形成的忙等还会在等待期间持续占用 CPU。

要避免忙等，通常需要条件变量（Condition Variable）补上等待-通知能力。它通常与一把互斥锁绑定：等待方在条件不满足时释放锁并进入休眠；通知方在改变条件后唤醒等待者；等待者被唤醒后会在重新持有锁的前提下继续执行。Go 中可以用 `sync.Cond` 表达这类关系：代码中的 `Wait()` 对应“释放锁并等待”，`Signal()` 与 `Broadcast()` 对应“条件变化后发出通知”。

使用 sync.Cond 实现等待-通知（示意）

```
1 type TaskQueue struct {
2     Items    int
3     Capacity int
4     mu       sync.Mutex
5     notEmpty *sync.Cond
6     notFull  *sync.Cond
7 }
8
9 func NewTaskQueue(capacity int) *TaskQueue {
10    q := &TaskQueue{Capacity: capacity}
11    q.notEmpty = sync.NewCond(&q.mu)
12    q.notFull = sync.NewCond(&q.mu)
13    return q
14 }
15
16 func (q *TaskQueue) Take() {
17    q.mu.Lock()
18    for q.Items == 0 {
19        q.notEmpty.Wait()
20    }
21    q.Items--
22    q.notFull.Signal()
23    q.mu.Unlock()
24 }
25
26 func (q *TaskQueue) Give() {
27    q.mu.Lock()
28    for q.Items == q.Capacity {
29        q.notFull.Wait()
30    }
31    q.Items++
32    q.notEmpty.Signal()
33    q.mu.Unlock()
34 }
```

使用 `sync.Cond` 时，有三点同步语义需要特别注意。第一，`Wait()` 会在内部以原子方式释放互斥锁并休眠，避免先解锁再休眠之间的竞态窗口。第二，`Signal()` 只唤醒一个等待者，`Broadcast()` 会唤醒所有等待者，但可能造成惊群效应。惊群效应是指许多等待者被同时唤醒后

一起竞争同一把锁或同一个资源，但其中大多数被唤醒后仍然无法继续执行，只是增加了调度和锁竞争成本。第三，改变条件必须在互斥锁保护下完成；发送通知（Signal 或 Broadcast）在 Go 中允许在锁外调用¹⁴。工程上常在持锁修改条件后发出通知，使从状态变化到通知等待者的链路遵守同一套同步协议；但这仍不保证被通知者会先于其他竞争者获得锁，因此等待者必须在返回后复检条件。若需要支持最多等待一段时间，`sync.Cond` 本身不提供超时能力，通常需要配合计时器或用 `channel + select` 来组织。更关键的是，等待通常要写成 `for` 循环复检条件，而不是用 `if` 做一次判断就继续执行。

一种常见误解是认为被唤醒就意味着条件已经成立。实际上，条件变量只负责唤醒等待者，并不保证目标条件在它重新获得锁时仍然成立；等待者必须再次检查条件。

唤醒后的竞争是最容易被忽略的一类情况。仍以任务队列为例，假设当前 `Items==0`，两个消费者的 goroutine 都在 `Wait()` 上休眠。此时某个生产者放入 1 个资源，并调用 `Broadcast()` 唤醒所有等待者。两个消费者都会被唤醒，但它们不会同时继续执行，而是要一个个重新抢到同一把互斥锁。第一个抢到锁的消费者把资源取走，使 `Items` 重新变为 0；第二个抢到锁的消费者如果用 `if`，就会假设唤醒即意味着条件成立，从而在空池上继续执行，轻则读到不存在的数据，重则把计数减成负数，破坏规则正确性。用 `for` 循环复检，才能让第二个消费者看到条件已不成立，然后回到等待状态。图 27 展示了这个最小竞争场景：唤醒只是重新检查条件的开始，不是继续执行的许可。

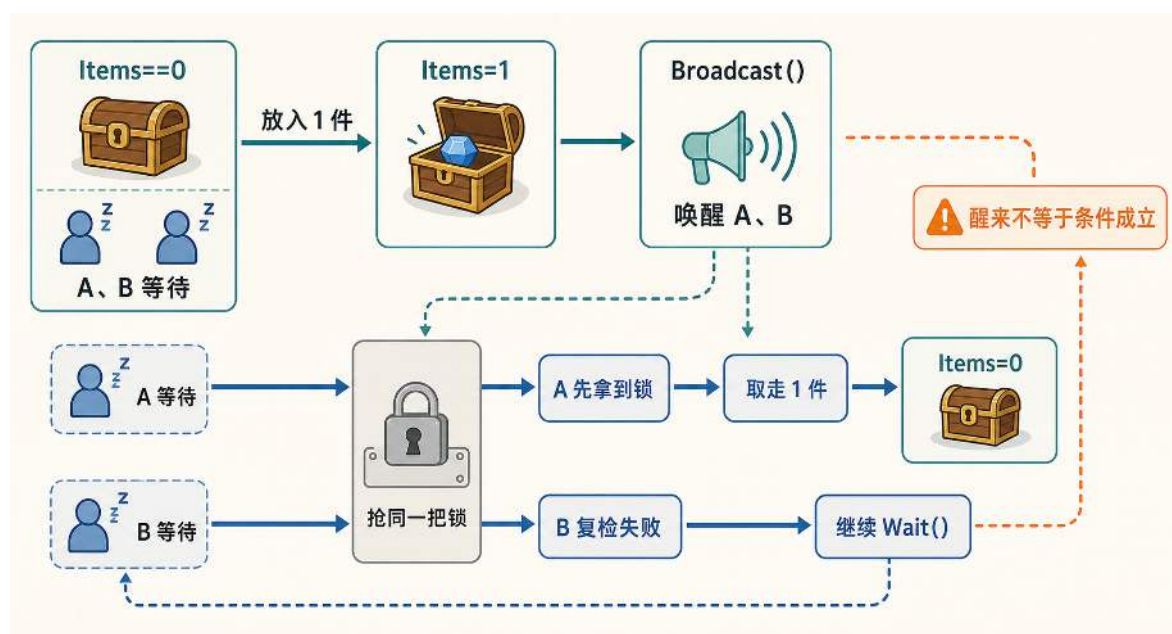


图 27: Broadcast() 唤醒后仍需复检条件。

有两类情况容易混淆。一些条件变量接口的规范允许等待者在没有显式通知时返回，这通常称为虚假唤醒；Go 的 `sync.Cond` 不采用这一语义，官方文档明确说明，`Wait()` 只有在收到

¹⁴Go `sync.Cond` 文档指出 `Signal` 和 `Broadcast` 允许但不要求调用者持有锁，见 <https://pkg.go.dev/sync#Cond.Signal>。

Signal() 或 Broadcast() 后才会返回。¹⁵ Go 仍然要求使用 for 复检条件, 原因不是 Wait() 会无通知返回, 而是通知并不等于为当前等待者预留了资源。从收到通知到重新获得互斥锁之间, 其他 goroutine 可能已经改变状态或消耗了可用资源; 上面的竞争场景已经足以说明, if 并不可靠。

因此, 条件变量的正确用法是每次唤醒后都重新检查条件。拿锁, 检查条件, 不满足就 Wait(); 被唤醒后再次检查, 直到条件成立才继续执行。值得强调的是, Wait() 返回时已经重新持有互斥锁, 因此复检条件与随后操作仍处在同一把锁保护的临界区内。

3.2.4 锁的粒度：性能与复杂度的折中设计

虽然互斥锁解决了并发环境下的正确性问题, 但条件变量的讨论已经暗示了一个趋势: 唤醒后仍要排队抢锁, 等待仍会传播为尾延迟。锁引入的核心性能成本是串行化。被锁保护的代码块在同一时刻只能被一个 goroutine 执行, 其他 goroutine 必须排队等待。这种串行化意味着并发度的下降; 当临界区很大或访问频率很高时, 锁竞争会成为系统瓶颈。因此在实际系统设计中, 锁的粒度选择需要在并发性能与实现复杂度之间权衡。

锁的成本来自两个层面。第一, 即使没有冲突, 获取与释放锁也需要维护锁状态并建立必要的内存同步, 这会让 CPU 付出固定开销。第二, 一旦发生冲突, 等待与唤醒会带来调度开销; 在多核下, 同一把锁对应的缓存行会在不同核之间来回转移, 产生缓存抖动。对延迟敏感的在线服务来说, 这些成本往往最先体现在尾延迟上: 平均耗时可能还看不出问题, 但偶尔一次排队就足以让用户感到卡顿。

因此, 用锁保证正确性只是第一步。更工程化的目标是把共享和竞争控制在可接受的范围内, 让规则确定的代价不会抵消系统的吞吐与稳定性收益。锁粒度的讨论, 就是在回答: 应该在哪里共享, 在哪里分隔, 才能既守住规则又守住性能。

锁粒度需要在实现简单和并发度之间取舍。全局锁把整个服务状态收进同一条队列, 实现最简单, 但无关操作也会互相等待; 区域锁按业务分片或空间分块收缩竞争范围, 使不同区域可以并发推进; 对象锁进一步把锁落到用户会话、订单、资源实例上, 并发度最高, 但跨对象操作会带来更复杂的加锁顺序和规则维护。图28用同一组用户与对象呈现三种策略的差异: 全局锁让所有人共排一条队, 区域锁让分区各自排队, 对象锁让每个实例各自排队。

¹⁵Go sync.Cond.Wait 的返回条件见 <https://pkg.go.dev/sync#Cond.Wait>。

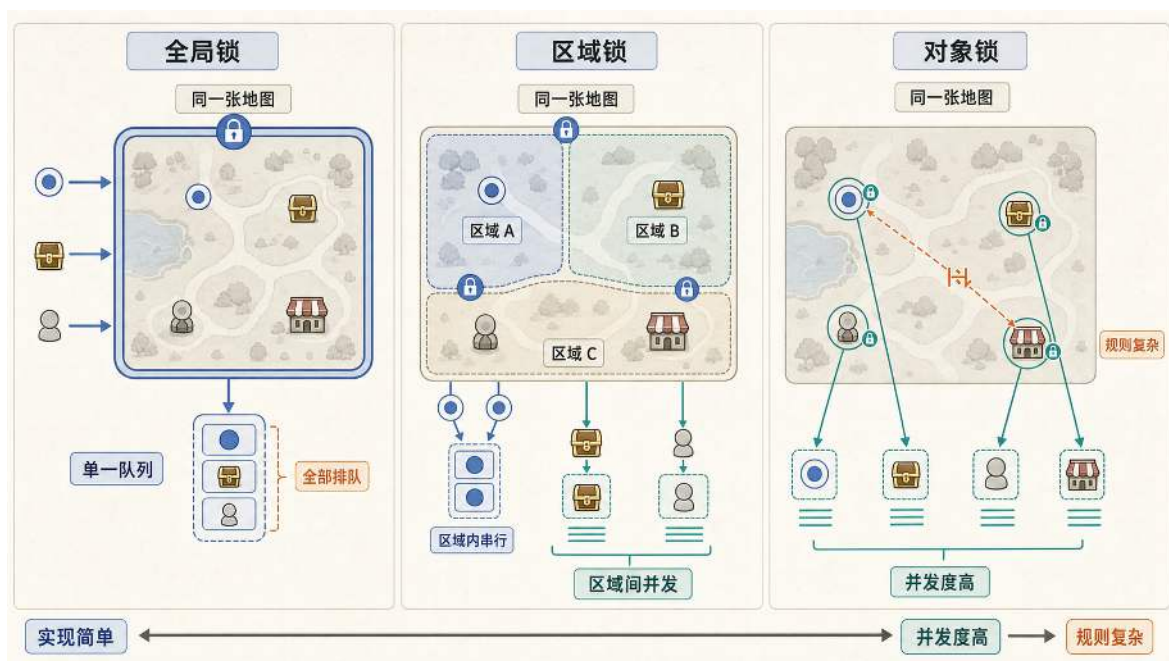


图 28: 锁粒度的三种策略对比：全局锁、区域锁与对象锁在性能和复杂度之间取舍不同。

全局锁是最直接的方案，整个服务器共用一把大锁，所有涉及共享状态的操作都必须获取这把锁才能执行。这种方式的优势是实现简单，死锁风险极低：只有一把锁，不存在多锁之间的获取顺序问题。然而，其代价是所有用户的操作都要排队等待，即使这些操作实际上访问的是完全独立的资源（如用户 A 读取自己的会话状态，用户 B 修改他人的订单记录），也必须依次执行。这会让核心路径接近单线程模型，抵消引入并发带来的收益。因此，全局锁更适合低频的全局配置、启动关闭等路径，在高频业务逻辑里要非常谨慎。

对象锁则走向了另一个极端，为每个独立的资源实例分配独立的锁。在在线服务中，这意味着每个用户的会话状态、每个订单、每个商品库存都由各自的 Mutex 保护。这样一来，用户 A 修改自己的配置时只锁定自己的会话，不必影响用户 B 查询商品列表的操作；两个用户同时操作不同订单时，也互不阻塞。对象锁通常能提供更高的并发度，让无关操作并行推进，但这会带来额外的管理和维护成本。锁的数量会增加，管理和维护成本会上升，也更容易遗漏某些应该保护的共享数据；当一个操作需要同时访问多个资源时（如转账需要同时锁定双方账户），还必须非常小心地设计加锁顺序，否则容易产生死锁。

区域锁在全局锁和细粒度锁之间取得折中，是实践中较为常见的选择。典型的做法是将数据按空间或业务域划分为多个独立区域（如按用户 ID 哈希取模、按地理区域分组），每个区域由独立的 Mutex 保护。区域内的操作需要串行执行，但不同区域之间的操作可以并发进行。当请求能够分散到多个区域，且一次操作通常只访问一个区域时，区域锁可以让不同区域并发推进，因而比全局锁少排队；它使用的锁数量又少于对象锁，生命周期管理也相对简单。相反，如果大多数请求集中在同一区域，或操作经常跨越多个区域，区域锁的并发收益就会明显下降。这种按区域划分负载的思想，与第 4.2 节要讨论的分布式分片存在连续关系：先在进程内把冲突局部化，

当单机承载接近上限时，再把同样的划分思想扩展到多机。

除了互斥锁的粒度选择，还可以根据读写比例选择读写锁。对于读多写少的数据结构，`sync.RWMutex` 允许多个读者并发进入，而写者保持独占，从而降低读路径上的锁竞争。另一类常见需求是限制某类操作的并发度，例如数据库查询或外部服务调用；这更像信号量，工程上常用带缓冲的 `channel` 来表达，容量就是允许的最大并发数。

如果共享状态非常简单，例如一个计数器、一个标志位，或者一次“只有当前值等于旧值时才写入新值”的状态转换，还可以使用原子操作。CAS (Compare-And-Swap, 比较并交换) 就是这类原语的典型形式：它先比较内存中的当前值是否仍等于预期值，若相等就原子地写入新值，否则返回失败，让调用方重试或走降级路径。Go 在 `sync/atomic` 中提供了这些接口。¹⁶原子操作适合保护很小的状态转换；一旦规则涉及多个字段、多个对象或需要维持复杂不变量，互斥锁、单写者事件循环或数据库事务通常更容易被审查和维护。

CAS 失败后通常需要重试，如果竞争很激烈，大量 goroutine 会在“读取旧值、尝试写入、失败再试”的循环里消耗 CPU，这就是自旋成本。另一个经典问题是 ABA：某个值先从 A 变成 B，又变回 A，CAS 只看当前值是否等于 A，可能误以为期间没有发生变化。工程上可以通过版本号、指针标记或更高层的锁与事务来避免误判。对在线服务而言，CAS 更适合计数器、状态位这类很小的局部状态；只要业务规则需要解释“谁先、谁后、为什么成功”，通常就应优先选择更清晰的锁、事件队列或数据库事务。

表2总结了不同锁粒度在在线服务中的典型应用场景与性能特点。在实际工程中，往往会混合使用多种粒度的锁，根据不同资源的访问特征选择合适的保护策略。

表 2: 不同锁粒度的应用场景与性能对比

锁的粒度	应用场景	并发性能
全局锁	服务器配置读取、全局公告	严重阻塞，接近单线程
区域锁	业务分片、地理分组	中等，区域间可并发
对象锁	单个用户会话、单个订单、 单个商品库存	较高，取决于锁顺序 与冲突分布

锁粒度的思路与后续的分布式分片存在连续关系。先在进程内把共享状态按对象或按空间划分成多个相对独立的域，让冲突尽量局部化；当单机的承载接近上限，再把这些域映射到不同的进程与不同的机器上，冲突处理也会从内存里的锁顺序扩展为网络上的状态交接与一致性协议。本章关心的是单机内如何把规则守住并把冲突收敛在局部；第 4.2 节和第 4.2.4 节会把同样的分解思想推到多机协作中。

本节讨论的 `sync.Mutex` 依赖进程内共享内存，只能保护单个进程中的数据。当同一份状态被多个服务器共同访问时，问题会从进程内同步转变为跨节点的状态归属、请求顺序与故障恢复，

¹⁶Go 原子操作接口见官方 `sync/atomic` 文档: <https://pkg.go.dev/sync/atomic>。

不能简单地把本地锁替换成一次远程操作。第 4 章将继续讨论这些机制；在本章的单机阶段，重点仍是通过合理的临界区与锁粒度，在正确性和并发性能之间取得平衡。

除了互斥锁和条件变量，Go 中还有三种常用机制可以补足并发控制：`defer` 保证锁释放和资源回收在函数退出时执行，带缓冲的 `channel` 可以充当信号量限制并发数，`select` 配合 `time.After` 可以为 `channel` 操作设置超时上限。这些机制也会在后面的连接池上限、等待超时和对象归还路径中再次出现。

至此，并发正确性已经从概念落到了具体工具：互斥锁保证临界区的唯一顺序，条件变量建立等待与通知关系，锁粒度决定串行化成本，`defer`、`channel` 信号量和 `select` 超时则补充资源释放、容量限制与退出路径。设计并发结构时，可以按不变量、同步关系、活性、可观测性的顺序检查这些机制是否真正成立。

首先写清必须始终成立的规则，例如同一个 NPC 只能送出一次宝物、同一个用户的关键消息必须按序处理、金币扣减不能丢失或重复。随后找出规则在哪里被检查、在哪里被修改，并确认这些动作是否处在同一套同步协议中。同步协议既可以是互斥锁，也可以是单写者事件循环、`channel` 所有权转移或串行事件队列；关键是让共享状态的每条读写路径都具有可依赖的先后发生于关系。

同步规则还必须保证系统能够持续推进。多把锁是否形成环等待，低优先级请求是否可能长期饥饿，队列是否有容量上限，等待是否能被超时或取消收住，连接退出后是否仍有 `goroutine` 泄漏，都会决定系统在压力下是否仍然可用。最后还要为这些判断提供可观测信号，例如锁等待时间、队列长度、`goroutine` 数和 P95/P99 延迟。规则被守住且运行状态能够被量化之后，后续瓶颈才会更清楚地落在连接、内存与数据访问等资源路径上。

3.3 单机资源优化与性能验证

到这里，单个服务实例内并发处理的两个基础问题已经得到解决：用并发结构隔离连接与 I/O 等待，用同步原语保证共享状态修改具有确定结果。接下来瓶颈会转向资源利用本身，例如连接的创建与销毁、临时对象的分配与回收，以及数据访问路径带来的延迟抖动。本节从连接、内存和数据访问三个方面讨论资源复用，再用压测与观测指标验证优化是否真正提高单个应用实例的承载能力。这里的“单机优化”强调应用实例尚未拆分到多台计算节点；数据库或缓存可以是独立进程或外部服务，正文关注的是当前实例如何控制访问它们的连接、并发和数据路径。

3.3.1 外部资源连接池：从短连接到连接复用

前面讨论的客户端连接，重点是稳定保活、及时读写和断线回收。在线服务访问数据库、缓存服务或其他后端组件时，也需要建立 TCP 连接，这里的连接管理转向服务与外部资源之间的访问路径。这类外部资源连接不应当随用随建，而应该通过连接池复用和限流。

每次建立 TCP 连接都需要完成三次握手过程，这意味着至少一次网络往返，还需要分配内核资源（例如套接字缓冲区、连接控制块等）并在用户态与内核态之间多次切换。以常见环境的数量级估算，局域网下握手往返通常在毫秒级，广域网可能到几十毫秒。如果在每次数据交互时

都重新建立连接，并在交互完成后立即关闭，端到端延迟会被握手与挥手拖长，系统调用与内核资源管理也会成为明显开销。

在高并发场景下，缺乏连接复用会让每次访问都重复支付建连与断连成本。以每秒 10,000 次数据库访问为例，如果每次访问都建立新连接，系统就要反复执行 TCP 握手、系统调用和内核对象的分配释放。单次开销可能不大，但大量请求叠加后会形成更长的排队与尾延迟。

频繁断连还会持续占用临时端口和连接状态。TCP 主动关闭方通常会在一段时间内保留 TIME_WAIT 状态，避免延迟到达的旧数据包干扰后续连接。创建和销毁速度过高时，新连接可能因端口或内核连接资源不足而失败。对本章而言，关键结论是：短连接把固定的协议与内核管理成本重复放大，连接池则把这些成本摊到一组可复用连接上。

连接池（Connection Pool）机制正是为了解决这一问题而设计的资源复用方案。连接池的核心思想是维护一组可复用的长连接：业务逻辑需要访问外部资源时，从池中借出空闲连接；池中如果没有空闲连接且未到上限时按需新建。使用完成后，连接不会被销毁，而是归还到池中等待下次复用。只有当池中所有连接都在使用且达到配置的最大连接数上限时，新请求才会等待，直到有连接被归还，或者在上层超时退出。Go 标准库的 `database/sql` 内置了这类池化能力。示例用“上限、空闲量、生命周期”三个配置说明资源约束如何落到程序中。

连接池上限与生命周期（示意）

```
1 func initDB(ctx context.Context) (*sql.DB, error) {
2     db, err := sql.Open("postgres", dsn)
3     if err != nil {
4         return nil, err
5     }
6
7     db.SetMaxOpenConns(100) // 最大打开连接数
8     db.SetMaxIdleConns(10)  // 最大空闲连接数
9     db.SetConnMaxLifetime(time.Hour) // 连接最大生命周期
10
11     if err := db.PingContext(ctx); err != nil {
12         db.Close()
13         return nil, err
14     }
15
16     return db, nil
17 }
```

示例中的三个配置量分别对应连接池的三个机制约束：总连接上限、空闲连接保留和连接生命周期。`MaxOpenConns` 限制同时打开的最大连接数，防止数据库服务器因连接过多而过载；当并发请求数超过这个限制时，多余请求会进入等待队列。`MaxIdleConns` 控制空闲连接的保留数量，使短时间内的新请求可以复用已有连接而不必重新握手。`ConnMaxLifetime` 为连接设置最大

存活时间，避免长期运行的连接累积状态异常或资源泄漏。通过这样的配置，连接数的膨胀被上限收住：无论瞬时并发有多大，系统最多只会维持 `MaxOpenConns` 条打开连接，其余请求要么排队等待，要么在上层触发超时与降级。握手与端口压力也因此从随负载失控增长，变成可配置、可观测的上限。

连接池上限并不是越大越好。`MaxOpenConns` 本质上把数据库可承受的并发显式化：设得过大，拥塞更容易被推到数据库端，查询排队与抖动会加重；设得过小，等待会被推回应用端，吞吐被过早截断。合理的上限通常需要结合数据库承载、单次查询耗时与尾延迟目标，通过压测与观测来确定。

短连接的问题在于每次请求都重复支付建连、查询、断开和 `TIME_WAIT` 的成本；连接池则把这些成本推到一组可复用连接上。图29对比了短连接模式与连接池模式：随用随建的线性路径每次请求都重复握手与断开；借出、归还与上限控制形成的循环路径则把连接复用起来，将成本收敛为可配置的资源管理。池子的上限不会让数据库容量变大，但它能把连接膨胀收敛为一个可配置、可观测的连接数上限。

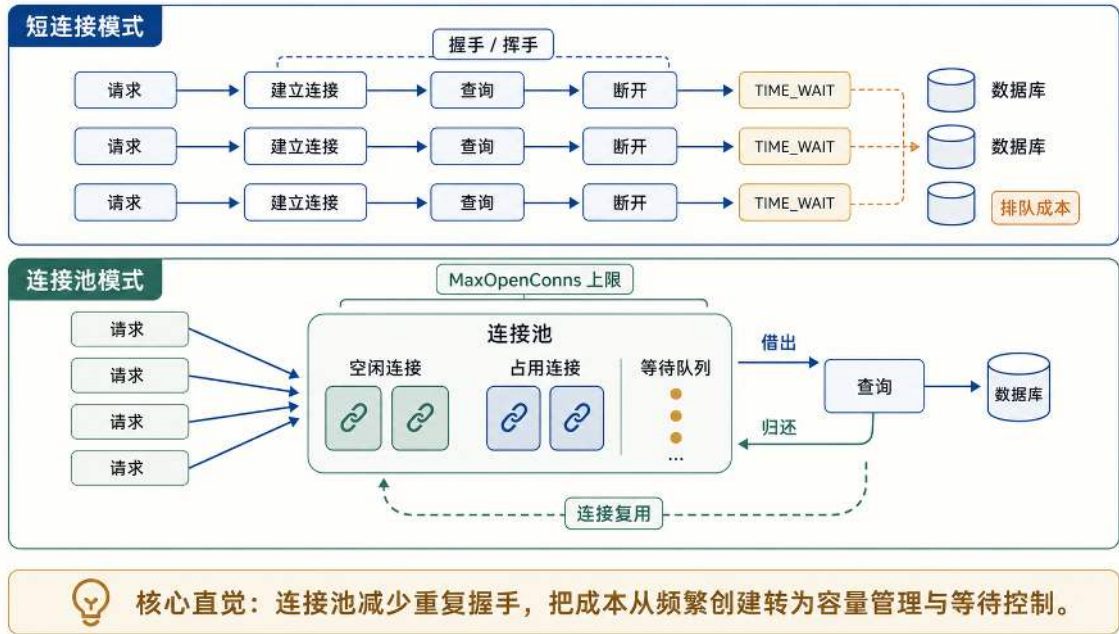


图 29: 连接池通过借出、归还和上限控制，把频繁建连变成可复用的资源循环。

连接池的关键循环很简单：有空闲连接就借出，没有空闲连接且未到上限就新建，达到上限就让请求等待。业务逻辑完成数据库操作后通常会调用 `Close()`（例如关闭查询结果集 `sql.Rows`），但在连接池语义下，这一步往往意味着归还连接，而不是真正关闭 TCP 连接。也正因为上限会带来等待，连接池必须配合超时、等待时间观测和数据库端指标一起使用，不能只把数值调大。

连接池的思路可以概括为：稀缺资源不要随用随造，而是放进一个有上限的池子里，按需借出，用完归还。这类做法可以统一称为池化（Pooling）。池化并不会凭空增加资源总量，它改变的是资源生命周期：把高频创建和销毁，改成可复用、可限额、可观测的循环。云计算里的很多资源管理也沿着这条思路展开，例如虚拟机资源池、容器镜像缓存池、负载均衡器后端池。从本

章的角度看，连接池是在单机内部先练习资源池化：把不可控的创建成本，变成有上限、可观测、可调参的资源管理问题。

3.3.2 对象池：复用短命对象以缓解 GC 压力

除了外部连接资源，程序内部的临时对象分配也可能成为性能瓶颈。高并发服务的网络与序列化路径会频繁创建字节缓冲区（`[]byte`）、请求上下文、编码对象和日志字符串。这些对象的生命周期通常只有几毫秒到几百毫秒，使用完成后很快成为垃圾。

在高并发场景下，这类对象的分配和回收频率可能达到每秒数次甚至数十万次，对 Go 运行时的垃圾回收器（GC）造成压力。Go 的垃圾回收器通过并发标记等机制缩短全局暂停，但 GC 并非零成本：标记、扫描、写屏障和必要的短暂停顿都会消耗 CPU，并可能影响缓存局部性。具体影响取决于 Go 版本、堆大小、对象存活率与分配速度，必须通过 GC 暂停、分配率和 P99 延迟等指标实际测量。

对象池把连接池建立的池化思路从外部资源延伸到进程内部：把频繁创建、很快丢弃的临时对象，改成“借出、清理、归还、再次复用”的循环。Go 标准库中的 `sync.Pool` 可以作为这种短命对象缓存的实现。¹⁷当业务代码需要临时对象时，优先从池中取出已有对象；使用完成后，将对象归还到池中而非立即丢弃。通过减少分配次数，对象池可以降低 GC 的触发频率和工作负载。

短命缓冲区的复用路径（示意）

```
1 var bufferPool = sync.Pool{
2     New: func() interface{} {
3         return new(bytes.Buffer)
4     },
5 }
6
7 func handleMessage(data []byte) {
8     buf := bufferPool.Get().(*bytes.Buffer)
9     buf.Reset()
10    defer bufferPool.Put(buf)
11
12    buf.Write(data)
13    processData(buf.Bytes())
14 }
```

这段示例把对象复用拆成三个保证复用正确性的约束。第一，取出的对象在再次使用前必须清理状态，避免受到上一次请求遗留数据的影响。第二，归还对象之前，不能让 `buf.Bytes()` 这样的底层切片继续被外部持有，否则对象回到池里后可能被下一次请求改写。第三，池中的对象

¹⁷`sync.Pool` 的语义限制见 Go 官方文档：<https://pkg.go.dev/sync#Pool>。

不保证一直存在，Go 运行时在 GC 时可能清空池，因此创建函数仍然是必要的兜底，不能假设池中永远有可复用对象。

这种对象池的价值在于把分配与回收的线性生命周期，改造成缓存与复用的循环模式。对于频繁创建且生命周期很短的对象，复用可以显著降低分配率，从而降低垃圾回收（GC）的触发频率与工作量，减少延迟抖动。不过，对象池有其适用边界：它只适合缓存短命对象，不适合作为长期持有的状态存储；需要跨请求保留的状态，仍应使用普通数据结构或专门的缓存系统。

3.3.3 缓存分层架构：热数据与冷数据的协同治理

连接池解决了外部连接的创建与销毁开销，对象池缓解了短命对象的分配与回收压力。至此，单机性能优化还剩最后一个资源维度：数据访问路径本身。在线系统中的数据访问模式存在显著差异：会话状态、热点计数和实时视图会被高频读写，要求较低访问延迟；账号、订单、历史记录和配置等数据访问频率可能较低，但对持久性和一致性要求更高。以游戏为例，玩家位置、在线状态、历史成就和装备配置分别对应上述不同的数据类型，但同样的划分也适用于电商的商品详情与库存、社交的好友动态、模型服务的热点参数等场景。

如果所有热数据都直接打到磁盘数据库，缓存未命中、索引扫描、锁等待或真实磁盘 I/O 都可能把查询拖进队列；即便数据库命中 buffer cache 或操作系统页缓存，高并发下也仍会消耗连接、CPU 与锁资源。反过来，如果所有数据都放在内存中，访问速度虽然快，但内存成本远高于磁盘，断电后也无法满足长期恢复要求。因此，单机阶段同样需要分层存储架构，让不同特征的数据落到合适介质上。

高频热数据：低延迟视图 热数据是指在在线服务运行过程中被高频率读取或修改的数据，这类数据对响应延迟极度敏感。典型场景包括用户会话状态、实时在线人数、热点商品库存、实时排行榜以及限流计数器。这些数据都读写频繁、数据量相对较小，但“能不能丢”的语义并不相同：用户会话短时丢失可能由客户端重新建立，限流计数器短时丢失会影响风控精度，排行榜则往往需要能从权威事件重新构建。

这类热路径通常会放到 Redis 这类内存存储中，使读写避开磁盘路径，优先满足低延迟访问。具体实现中可以通过客户端库访问 Redis，也可以使用批量操作减少往返次数；但这些工具细节服务的仍是同一个机制目标：把高频、低延迟、可重建或可补偿的数据放到更快的介质上。为了降低数据丢失风险，可以配置 Redis 的 AOF（Append Only File）持久化机制，将写操作记录到磁盘日志，在服务器重启后重放日志恢复数据。不过 AOF 也不等于零丢失，具体保障取决于刷盘策略、故障时机与恢复流程。

低频冷数据：权威持久记录 冷数据是指访问频率较低、更新不频繁，但对数据完整性和持久性要求较高的数据。典型场景包括用户注册信息、历史成就记录、基础属性配置、交易历史以及全局配置。这类数据的共同特征是访问频率低、数据量可能很大、必须保证持久性和可恢复性。

这类数据更适合落在 PostgreSQL 这类关系型数据库中，由数据库事务、约束和持久化日志保护权威记录。正文关心的重点不是某个 ORM 如何映射模型，而是哪一层保存最终事实：冷数

据通常可以在 Redis 中保留一份副本用于加速读取；当 Redis 缓存失效或服务器重启时，再从 PostgreSQL 加载权威数据恢复视图。实验代码中可以使用 gorm 这类 ORM 操作数据库，相关表结构和实验入口见附录 B。

Redis 与 PostgreSQL 的分层并不难，难点在于写清两层的职责分工。在两层架构中，Redis 位于低延迟热路径，PostgreSQL 保存权威数据与持久化记录，两层之间的同步关系主要落在三条跨层路径上。路径 A 处理热数据读取：先访问 Redis，未命中时回源 PostgreSQL 并将结果回填缓存。路径 B 处理关键写入：先落到 PostgreSQL 的事务或持久化日志，确保权威记录可恢复。路径 C 处理失效更新：当权威数据变更后，删除旧缓存或更新 Redis 中的派生视图，避免长期返回旧结果。这样，Redis 承担低延迟访问，PostgreSQL 守住持久性和一致性，两层之间的同步规则也不会退化成含糊的“双写”。

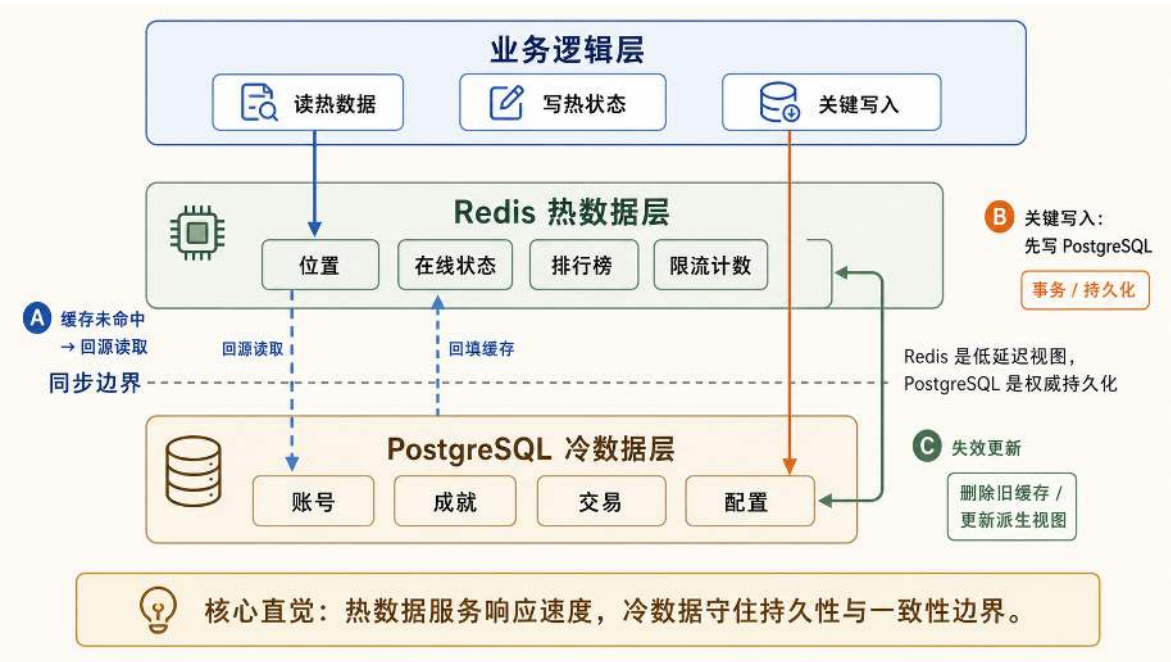


图 30: Redis 承接低延迟热路径，PostgreSQL 保存权威数据；跨层路径要分别处理未命中、关键写入和失效更新。

缓存容量达到上限后，系统必须决定哪些键先被移出，这就是缓存淘汰策略要解决的问题。最常见的两类策略是 LRU（Least Recently Used，最近最少使用）和 LFU（Least Frequently Used，最不经常使用）。LRU 假设最近访问过的数据短期内更可能再次被访问，因此优先淘汰最久没有访问的键。LFU 更看重访问频率，倾向于保留长期高频访问的数据。¹⁸ 在线服务里的会话状态、排行榜与配置缓存访问模式不同，淘汰策略也不应一概而论。临时状态可以设置较短 TTL，让过期机制主动释放内存；全局配置更适合通过版本号或失效通知刷新，而不是完全依赖容量淘汰。

¹⁸ 页面置换与缓存淘汰算法的经典研究可追溯到 Belady 1966，见 L. A. Belady, “A Study of Replacement Algorithms for a Virtual-Storage Computer”, IBM Systems Journal, 1966, <https://doi.org/10.1147/sj.52.0078>。

缓存还会遇到三类经典失效问题。第一类是缓存穿透：请求访问的键本来就不存在，如果每次都绕过缓存访问数据库，恶意请求或异常参数会把压力直接推到后端。常见处理方式是在入口做参数校验，并对“确定不存在”的结果设置短 TTL 的空值缓存。第二类是缓存击穿：某个热点键突然过期，大量请求同时回源，数据库会在短时间内承受集中访问。处理方式可以是互斥回源、提前刷新或逻辑过期。第三类是缓存雪崩：大量键在同一时间失效，导致回源流量集中爆发。处理方式通常是给 TTL 加随机抖动，分散过期时间，并为回源路径设置限流与降级。这三类问题说明，缓存分层不只是把数据放进 Redis；它还要把未命中、回源、淘汰和失效传播做成可控流程。

数据同步策略的选择 在分层存储架构中，Redis 和 PostgreSQL 之间需要明确的数据同步策略，以说明哪一层是权威数据，哪一层只是加速视图。不同的同步模式在一致性保证、性能开销和适用场景上存在差异。

对于关键资产操作，最稳妥的做法是把结果先写入 PostgreSQL 的事务或持久化日志，确保权威记录可恢复，再可靠地更新 Redis 视图。这种“权威写入 + 缓存刷新”模式让最终事实和恢复依据最清楚，但缓存刷新仍可能失败，因此需要可重试的更新或失效机制。该模式的写路径较长，适用于账户转账、订单支付等不容出错的场景。

写穿（Write Through）模式要求每次写操作同时更新 Redis 和 PostgreSQL，写入操作必须等待两个存储系统都确认成功后才返回。它能降低读到旧数据的概率，但 Redis 与 PostgreSQL 的双写本身并不是一个天然的原子事务；如果中间失败，仍然需要重试、幂等键、补偿任务或以数据库事务/事件日志作为权威来源。

写回（Write Back）模式先将数据写入 Redis 并立即返回，随后通过异步任务定期将 Redis 中的数据批量刷入 PostgreSQL。这种模式的写入延迟更低（通常在毫秒级），吞吐量高，但存在数据丢失风险：如果 Redis 在数据尚未刷入数据库时发生故障，未持久化的数据会永久丢失。写回模式适用于允许短时数据丢失的场景，如用户位置同步、临时会话状态、计数器更新等。

失效更新（Cache Aside）模式在写入时只更新 PostgreSQL，并删除 Redis 中的对应缓存。下次读取时，发现 Redis 中没有数据，则从数据库加载并回填到 Redis。这种模式适合读多写少的场景，如静态配置数据（商品属性、系统参数、区域配置等）、历史记录查询等。由于配置数据很少变化，大部分读取都能命中 Redis 缓存，只有在配置更新后的第一次读取会访问数据库。

表3总结了四种数据同步策略的特征与适用场景。在实际工程中，不同类型的数据会采用不同的同步策略。关键资产先保正确性，热路径再谈速度；能丢、能补、能重建的数据，才有空间换取更低延迟。

表 3: 数据同步策略的特征与适用场景

同步策略	一致性/性能	适用场景
权威写入 + 缓存刷新	权威来源清晰, 缓存可能短时陈旧, 写路径较长	账户转账、订单支付等关键资产操作
写穿 (Write Through)	缩短缓存陈旧窗口, 部分失败仍需修复, 写延迟较高	可补偿的同步视图、低风险状态同步
写回 (Write Back)	最终一致, 写延迟低	用户位置、临时状态等允许短时丢失的数据
失效更新 (Cache Aside)	最终一致, 读性能优	商品配置、系统参数等读多写少的静态数据

在实际工程中, 不同操作会根据数据特征选择不同的同步策略。以在线服务为例, 用户位置或会话心跳是最高频的操作, 每秒可能触发数十次更新, 但丢失最近几秒的数据通常影响较小: 服务端可以回滚到最近一次权威快照, 或根据客户端重新上报重新计算。这里要守住服务端权威模型, 客户端上报只能作为输入线索, 必须经过校验规则, 不能直接当作可信状态。因此, 这类高频低敏数据可以采用写回策略, 先写入 Redis 保证低延迟响应, 再由后台任务每隔数秒批量刷入持久存储。

交易与支付事件频率远低于位置更新, 但它们会影响账户余额与订单状态。更稳妥的做法是把交易事件先写入权威事件日志或 PostgreSQL 订单表, 再用 Redis 视图作为可重建的派生缓存, 而不是把“双写成功”当作天然一致。账户余额同理, 作为核心资产, 通常应以数据库事务、账本或事件流作为权威来源, 再可靠地刷新缓存。商品配置、系统参数等静态数据属于典型的读多写少场景, 适合采用失效更新策略: 首次读取时从数据库加载并缓存到 Redis, 后续读取直接命中缓存; 只有在运营人员修改配置时才删除缓存, 触发下次读取重新加载。示例代码中各操作与缓存策略的对应关系, 可结合附录 B 的实验指南阅读。

这种分层存储架构不仅是技术实现的优化, 也是成本与可靠性的工程权衡。Redis 按内存容量计费, 成本较高但延迟更低; PostgreSQL 以磁盘存储为主, 成本更低但延迟更高。通过将热数据与冷数据分离, 可以在满足性能需求的前提下, 把内存规模控制在合理范围。以下数字只用于建立数量级概念, 不是通用工程标准: 以一个 10 万在线用户的实时服务为例, 如果每个用户会话都直接放在 Redis 中, 内存需求可能达到 10GB 量级; 而在分层架构下, Redis 只承载在线状态、当前位置等少量热数据, 单会话只保留 KB 级别即可, 把内存需求压到百 MB 量级, 其余冷数据仍落在 PostgreSQL 上。把性能与成本同时纳入设计目标, 是单机阶段优化能够落地的重要前提。

3.3.4 性能验证：用可观测指标量化优化效果

上述从连接池、对象池到缓存分层的优化设计，都是为了在不拆分应用实例的前提下争取更高的并发承载能力。这些优化最终需要由压测曲线验证：吞吐是否上升，P99 是否下降，CPU、GC、锁等待和连接等待是否转移到新的瓶颈位置。只有建立了可观测的性能基线，才能判断继续优化单机代码是否还有收益，何时继续纵向优化，何时转向第 4 章的横向扩展。到了第 5.3 节，这些指标还会进一步变成弹性伸缩的输入信号。

性能验证的核心在于定义和测量关键指标。对于在线系统，最重要的三类指标是吞吐量、延迟分布和资源占用。

吞吐量 (Throughput) 通常用 QPS (Queries Per Second) 或 TPS (Transactions Per Second) 来衡量，表示系统每秒能够处理的请求数量。对本章的在线服务，可以测量每秒处理的客户端请求数、规则计算数、状态推送数等。吞吐量反映系统的整体承载能力，是评估优化效果最直观的指标。

延迟分布 (Latency Distribution) 关注每个请求从发起到收到响应的耗时。与平均延迟相比，P95 和 P99 延迟（即 95% 和 99% 的请求在多长时间完成）更能反映用户的真实体验。对于实时服务，例如平均延迟只有 10 毫秒级，但 P99 延迟达到 500 毫秒级，意味着每 100 个请求中约有 1 个会遭遇明显卡顿；由于每个用户在一次会话中会发出大量请求，多数活跃用户都可能在某些时刻遇到这种延迟峰值。

资源占用 (Resource Utilization) 包括 CPU 使用率、内存占用、网络带宽消耗以及 GC 统计信息。这些指标用于定位瓶颈所在：如果 CPU 使用率长期接近 90% 但内存充足，说明瓶颈更可能在计算；如果 GC 暂停频繁或分配率过高，说明内存分配模式需要优化；如果网络带宽打满，说明需要优化推送范围、增量消息或序列化路径。

指标能指出哪里不对，但要回答为什么，必须把现象映射到更具体的观测手段。表 4 给出一条常用的定位路径：先看信号，再选工具，最后用细粒度指标归因到代码或结构。

表 4: 从现象到工具：常见瓶颈的定位路径

现象/信号	常见原因	观测手段	重点指标
吞吐低但 CPU 很高	算法慢、热点函数集中	pprof CPU profile	火焰图、Top 函数
延迟抖动且 GC 频繁	分配率高、短命对象多	pprof heap 与分配统计	allocs/op、GC 暂停与频率
并发上来后明显变慢	锁竞争、阻塞等待	pprof mutex / runtime trace	阻塞时间占比、锁等待
goroutine 数持续增长	goroutine 泄漏、退出路径缺失	pprof goroutine	goroutine 数趋势、堆栈归因
请求延迟升高且连接池等待增加	连接池过小、数据库处理能力不足	sql.DBStats + 数据库监控	WaitCount/WaitDuration、活跃与空闲连接
缓存加速收益不稳定	命中率低、集中回源或网络往返升高	缓存指标 + Redis/dis/网络监控	命中率、回源比例、Redis RTT
网络吞吐接近上限	广播放大、序列化路径重	压测 + 网络监控	带宽、重传、发送队列

连接池等待指标用于判断连接复用是否只是把排队转移到了池内，缓存命中率与回源比例用于判断热点读取是否真正离开了数据库。它们应与吞吐和尾延迟一起观察，不能只用单次请求耗时判断优化是否有效。

量化优化效果时，需要设计对比实验。可以先用一个小型压测样本确定指标的大致数量级：这些数字只用于比较趋势，实际结果取决于硬件、网络与实现细节。场景可以设为模拟 100 个并发客户端，每个客户端每秒发送 5 次状态更新请求和 1 次业务计算请求，持续运行 5 分钟，记录吞吐量、P99 延迟以及资源占用。后续要评估数千连接或上万连接的承载上限时，可以沿用同一套指标，只是把并发规模逐级抬高。

对比实验的关键是控制变量：每次只引入一项优化，其余条件尽量保持不变。这样，当吞吐与尾延迟发生变化时，才能把变化可靠地归因到这项优化，而不是把多个变量叠在一起得到一组不可解释的数字。

表 5: 控制变量压测的示意性趋势

实验阶段	吞吐变化	P99 延迟变化	主要观察点
基线版本	偏低	偏高	建连、分配、数据库访问是否排队
加入连接池	可能上升	可能下降	连接等待、数据库端连接数
加入 <code>sync.Pool</code>	小幅变化	可能更稳定	分配率、GC 暂停、堆增长
加入 Redis 热路径	可能上升	可能下降	缓存命中率、回源比例、一致性补偿

在这个基准场景下，如果未引入连接池，频繁建连与断连会把时间花在握手与系统调用上，吞吐容易受限，尾延迟也更容易升高。引入数据库连接池后，复用机制吸收了握手开销，压测时应重点观察连接等待和 P99 延迟是否回落。

进一步引入 `sync.Pool` 减少短对象分配后，GC 触发频率与暂停抖动通常会下降，延迟曲线也可能更稳定。最后，当热路径迁移到 Redis 并采用两级存储策略后，在数据库访问未命中缓存、走磁盘路径时可达十毫秒量级，而 Redis 经网络访问通常在亚毫秒到低毫秒量级。具体差值取决于数据是否命中内存、网络距离、查询复杂度和当前负载，必须由压测结果确认。通过这样的对比实验，不仅能看到每项优化带来的方向性收益，也能识别出当前阶段的主导瓶颈。

更重要的是，性能验证能帮助判断继续优化单机代码是否还有收益。随着并发连接数继续上升，如果看到 CPU 长期接近饱和，而热点函数、锁等待、GC 与数据库访问都被逐项压过，继续微调代码只能带来很小的收益，就说明单机算力可能已接近上限。此时继续堆砌优化技巧的边际收益会下降。相反，把系统拆成多个实例并通过负载均衡分散请求，常常能继续提高承载能力，但提升幅度取决于状态拆分、跨服通信与共享存储瓶颈，并不天然线性。是否转向横向扩展，要比较两条路径的净收益：继续单机优化能否以合理投入满足容量目标，以及拆分带来的容量、弹性与可用性收益能否覆盖状态管理和跨节点协作成本。当继续单机优化的投入增加而容量收益已很有限，且可行的拆分方案能够带来足够收益时，才形成转向第 4 章分布式架构的信号。

3.4 小结：单机并发的机制与限制

本章的第一条主线，是用并发隔离等待传播。进程划分资源与故障影响范围，线程承接操作系统调度，goroutine 把大量等待型任务复用到较少线程上，channel 与 `select` 则把消息交接、超时和背压写进程序。

第二条主线，是用同步保护共享状态。库存扣减、余额转账、协作文档提交、限量抢购、NPC 奖励归属，本质上都要求检查与修改遵守确定顺序。`mutex` 可以保护临界区，条件变量可以补上等待与通知，锁粒度则决定正确性需要付出多少串行化成本。

第三条主线，是通过资源复用与数据分层降低单次操作的固定开销。连接池、`sync.Pool` 和分层存储分别减少反复建连、短命对象分配和慢速访问带来的成本，压测与观测指标负责验证这些优化是否真正改善吞吐、尾延迟和资源占用。

从本书的原型演进来看，本章完成的是单机优化阶段的主体工作：先明确单机内的并发潜力、资源复用方式和数据访问路径，再用压力测试和性能监控建立基线。如果增加单个实例的 CPU、内存或其他资源，才属于纵向扩展（Scale Up）。这样才能量化地回答几个问题：当前硬件能支撑多少并发请求或连接，每项优化带来了多大收益，主要瓶颈还位于哪里。

判断系统是否应转向横向扩展（Scale Out）时，不能只看某个固定阈值，而要综合观察一组信号。第一，CPU、网络带宽、连接数或内存占用长期逼近安全水位，继续加压会直接推高队列长度和 P99 延迟。第二，热点函数、锁等待、GC、连接池和数据库访问已经逐项优化，局部改动只能带来很小收益。第三，性能瓶颈开始集中在单机不可再分的资源上，例如单进程事件循环、单库写入、单一状态分区或单个网络出口。第四，系统已经能够说明按用户、对象、区域或服务职责拆分后的状态归属、拆分键和跨实例交接规则。当多项信号持续出现，且拆分带来的容量、弹性或可用性收益能够支撑额外的状态管理与跨节点协作成本时，第 4 章的分布式架构才成为可得出的工程选择。

第 4 章将从单台服务器进入分布式阶段。此时的核心矛盾会从单机内的并发效率，转向多个实例之间怎样分配请求、同步状态并在故障时继续协作。电商订单、协作文档、即时通讯、模型任务以及在线游戏中的跨服裁决和玩家迁移，都需要处理跨节点的数据同步、全局顺序与故障恢复。再往后，第 5.2.4 节会把这些服务放到集群编排环境中，第 6.1 节会进一步解释容器如何把进程、资源限制与隔离机制工程化。

3.5 思考题

1. 在顺序主循环里，服务器会依次从每个连接读取输入、推进逻辑、再推送状态。假设其中一个连接的读取偶尔阻塞到 500ms，而其余连接都在 10ms 内就绪。请用因果链解释为什么延迟会从一个连接传播到所有连接，并指出把收包、处理与推送拆开后，哪一段等待被隔离了。然后思考：当我们引入并发后，哪些共享状态最容易首先被写穿，从而把问题从性能转移为正确性。
2. 某个实现把每个连接的收包与处理拆成 goroutine，发现平均延迟变化不大，但 P99 明显下降；另一个实现把规则计算改为多核并行，单轮计算时间下降但 CPU 使用率上升。请分别判断这两种改造更接近并发还是并行，并说明它们各自主要缓解的瓶颈是什么。再思考：在多人在线场景里，什么时候并发比并行更优先，什么时候反过来。
3. 为了做数量级估算，假设操作系统线程需要预留 1MB 栈虚拟地址空间，而 goroutine 的初始栈为 2KB。若服务器要承载 20,000 个在线连接，分别用一连接一线程和一连接一 goroutine 组织读写路径，单从栈预留量角度看两者大约相差多少个数量级。这里计算的是理想化的预留量，不代表进程实际驻留内存；然后思考：除了栈空间之外，

还有哪些因素会限制并发规模，为什么这也是系统要可观测的原因。

4. 本章强调同步与异步描述的是接口语义，而阻塞与非阻塞描述的是调用行为。请把下面四种接口放到图24的四个象限中，并说明理由：`Read()` 一直阻塞到结果、`TryRead()` 立即返回 (`data, ok`)、`ReadAsync(cb)` 回调返回结果、`ReadAsync()` 返回句柄并允许 `Wait()` 等待。再思考：在线服务的收包路径和落库路径各适合哪些组合，为什么。
5. `channel` 常被当作连接内的事件队列。请结合多人在线的消息处理链路解释：缓冲区过小会把什么压力暴露到上游，缓冲区过大又可能把什么问题拖到更晚才爆发，从而放大尾延迟。然后思考：如果要在高压下避免无休止排队，超时与降级应该写在什么位置，并用哪些可观测指标验证它确实保护了系统而不是掩盖了问题。
6. 以 NPC 抢宝为例，请写出必须被维护的最小不变量（例如同一份宝物最多只会被领取一次），并指出检查条件与修改状态之间的竞态窗口位于哪里。随后说明互斥锁或单写者模型如何把这段关键区收敛成唯一顺序。在此基础上切换到容量有限的任务队列场景：当队列为空时消费者需要等待，当队列满时生产者也需要等待，为什么 `Wait()` 必须放在 `for` 循环里复检条件，才能让正确性不依赖侥幸的调度时机。
7. 假设 Go 没有提供 `sync.Cond`，请仅使用 `channel` 和 `select` 重新设计一个容量有限的任务队列：队列为空时消费者应如何等待，新任务入队后如何唤醒后续消费者，又如何处理超时或服务取消。请对比这种消息传递方案与“互斥锁 + 条件变量”方案在状态归属、多等待者唤醒、超时取消和实现复杂度方面的取舍。
8. 双对象操作是死锁的高发区。请以转账或交易结算为例，构造一个最小并发时序使两条路径形成环等待，并给出一种统一的加锁顺序规约来消除环。然后思考：如果事件处理引入优先级，为什么仍可能出现饥饿，怎样的最低保障策略能把等待时间收敛到可接受的范围内。
9. 本章引入了连接池、`sync.Pool` 与缓存分层三类优化。请设计一个控制变量的对比实验：每次只引入一项优化，记录吞吐、P95/P99、CPU、内存、GC 与阻塞时间等指标，并说明如何根据观测结果判断瓶颈主要来自 I/O 等待、锁竞争还是 GC 抖动。再思考：当单机优化的边际收益显著下降时，哪些信号最能支持转向分布式扩展的决策。